ECOLE TECHNIQUE
ECOLE DES MÉTIERS LAUSANNE

« PROPHOTOSTOCK »

Application mobile pour une banque d'image

ADRIAN TOLEDO

02 FÉVRIER 2026 13 MARS 2026

ADRIAN TOLEDO
Cheffe du projet : Suranutha Thirunavukarasu

Table des matières

1. Analyse préliminaire 5
1.1. Introduction 5
1.2. Objectifs 6
1.3. Planification initiale 7
2. Concepts 8
2.1. Analyse Fonctionnelle 8
2.2. Flux de navigation 9
2.3. Maquettes UI/UX 10
2.4. Modélisation des Données 14
2.5. Stratégie de test 15
2.6. Risques techniques 15
2.7. Planification 15
2.8. Dossier de conception 16
3. Dossier de réalisation 18
3.3. Description des tests effectués 20
3.4. Erreurs restantes 22
3.5. Liste des documents fournis 23
3.6. Analyses de temps réalisé 23
4. Conclusions 24
4.1. Objectifs 24
4.2. Points positifs 25
4.3. Points négatifs 25
4.4. Difficultés particulières 25
4.5. Suites possibles pour le projet 25
5. Annexes 26
5.1. Résumé du rapport du TPI 26
5.1.1. Contexte et mandat 26
5.1.2. Solution réalisée 26
5.1.3. Résultats principaux 26
5.1.4. Bilan et suites 26
5.2. Vocabulaire 27
5.3. Manuel d'Installation 28
5.4. Manuel d'Utilisation 29
5.5. Archives du projet 30
5.6. Sources – Bibliographie 31

5.7. Journal de travail 33

Table des illustrations

Graphique 1 Pourcentage des tâches planifiées 7
Graphique 2 Tâches planifiées 16
Graphique 3 Tâches réalisées 23
Graphique 4 Temps par tâches réalisées 24

Maquette 1 Couleurs 10
Maquette 2 Logo 11
Maquettes 3 Authentification 11
Maquette 4 Catalogue et Confirmer Contrat 12
Maquette 5 Publication d'image 12
Maquette 6 Contrats et Chiffre d'affaires 13
Maquette 7 Profil 13

Schéma 1 Structure de ProPhotoStock 5
Schéma 2 Objectifs avec Waterfall 6
Schéma 3 Cas d'utilisation 8
Schéma 4 L'architecture logicielle 9
Schéma 5 Flux de navigation 10
Schéma 6 Modèle Conceptuel de Données (MCD) 14
Schéma 7 Modèle Logique de Données (MLD) 14
Schéma 8 Architecture Mobile 17
Schéma 9 Arborescence Frontend 18
Schéma 10 Arborescence Backend 19

Figure 1 Container Docker 20
Figure 2 Dossier Doc 20
Figure 3 Formulaire de publication 30
Figure 4 Partager 30
Figure 5 Contrats 30
Figure 6 Confirmer Contrat 30
Figure 7 Catalogue 30
Figure 8 Se connecter 30
Figure 10 Tableau de Bord 31
Figure 9 Profil 31

1. Analyse préliminaire

1.1. Introduction

ProPhotoStock App est la future application mobile de la société ProPhotoStock, spécialisée dans la distribution de photographies pour les professionnels de la publicité, du graphisme et des médias numériques. Cette application a pour objectif de stocker et centraliser des données des images et de la mettre à la disposition des clients sous contrat, selon les droits d'image souhaités.

L'interface mobile affichera une expérience utilisateur fluide et pratique pour montrer une galerie de photos triées par genres et popularité. Depuis le smartphone, les clients peuvent acheter les droits d'image, en précisant le type contrat et son exclusivité. Pour les photographes, depuis la même application, ils auront la possibilité de partager leurs images directement prises sur l'appareil de photo ou depuis leur galerie d'images personnelles.

Le suivi des tous les contrats est géré système depuis l'application mobile. Chaque utilisateur disposera d'outils adaptés à son profil : le client peut voir tous ses contrats actifs et réaliser des nouveaux contrats, les photographes peuvent voir leurs ventes et leurs revenus, et les responsables de l'entreprise peuvent suivre en direct tous les contrats et les bénéfices générés par image ou photographe de l'application comme un rapport mensuel.

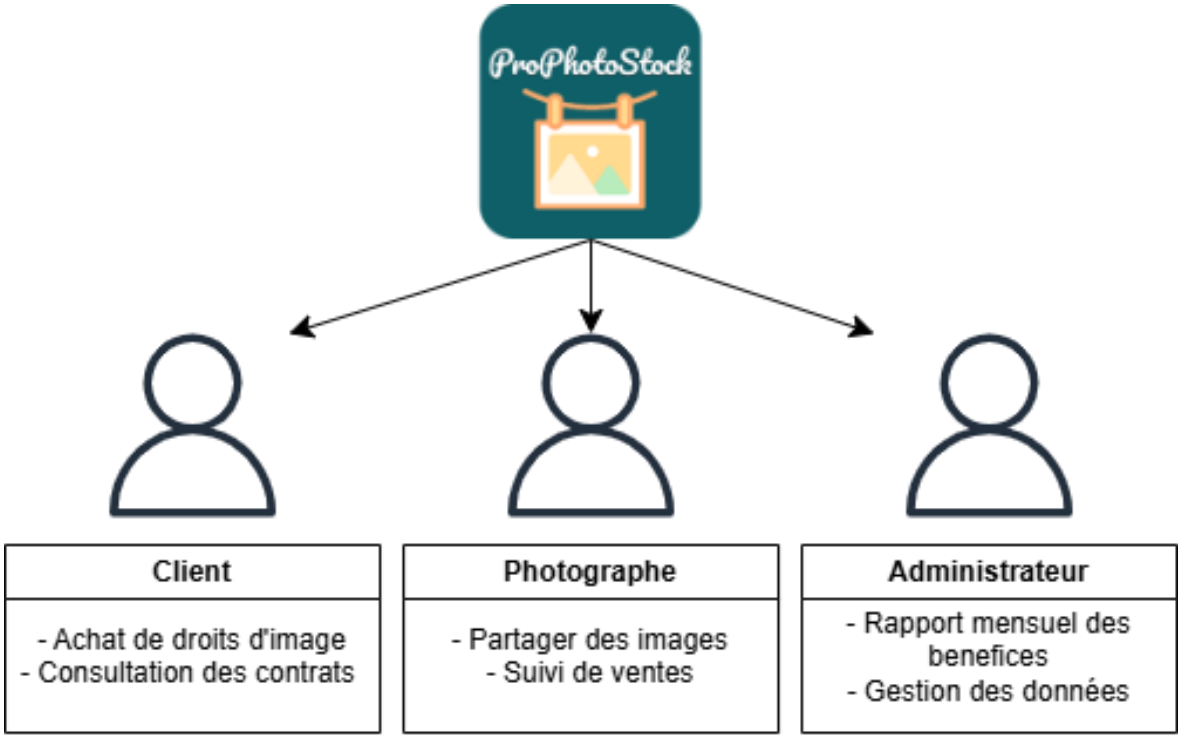


Schéma 1 Structure de ProPhotoStock

1.2. Objectifs

L'application ProPhotoStock doit réaliser une solution pour la gestion de photographies professionnelles, structurée avec des fonctionnalités suivantes :

- **Gestion des accès** : Implémenter un système d'authentification et sécurisé de création de compte selon les différents rôles d'utilisateurs, et montrer ces - informations personnelles.
- **Catalogue et filtrage d'images** : Développer un catalogue vertical permettant de visualiser toutes les images disponibles et de les filtrer par genre (Portrait, Paysage, Sport) ou trié par popularité.
- **Processus d'acquisition contrats** : Créer une système pour confirmer l'acquisition de droits d'image et générer des contrats numériques basés sur le type d'exclusivité choisi.
- **Contribution des photographes** : Photographes peuvent téléverser des images directement depuis la caméra ou la galerie de leur smartphone.
- **Tableaux de contrats personnalisés** :
  - **Clients** : Suivi des contrats actifs.
  - **Photographes** : Consultation des revenus générés par contrats.
  - **Entreprise** : Vue d'ensemble des bénéfices mensuels des contrats et des images.

Afin de garantir la livraison du projet dans le délai imparti de 90 heures, la méthodologie choisie est le modèle Waterfall (cascade). Sur une période de 5 semaines, je suivrai une planification afin de respecter les étapes classiques du cycle.

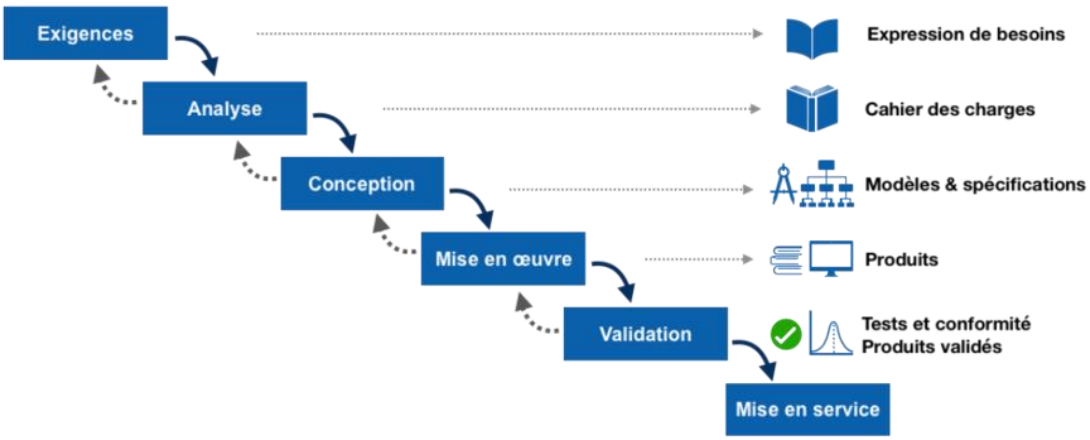
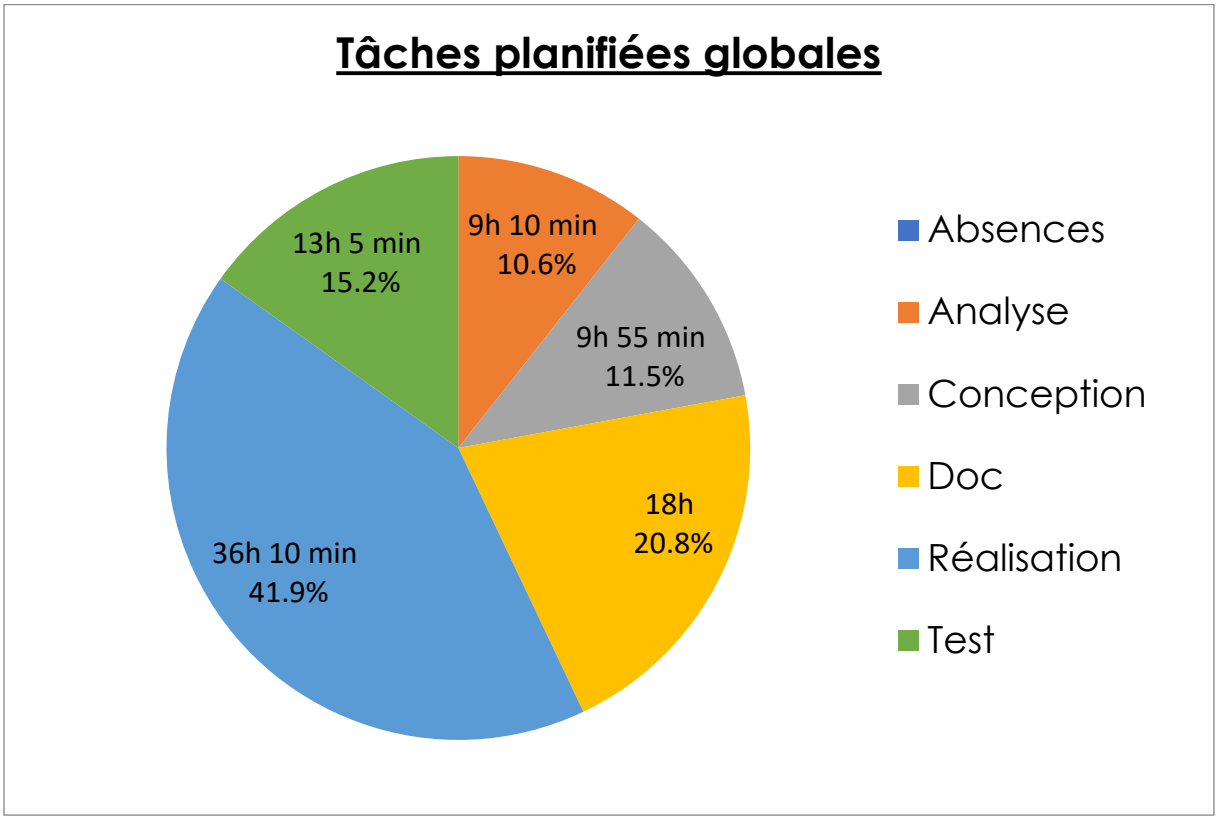


Schéma 2 Objectifs avec Waterfall

En parallèle, ce projet sera documenté dans le présent rapport au fur et à mesure de l'avancement du développement. Ce processus sera complété par un suivi quotidien via un journal de travail détaillé et une gestion de versions avec Git, permettant de montrer l'évolution du code à travers des commits.

1.3. Planification initiale

La planification de ce projet a été établie pour livrer un rapport et une démo fonctionnelle pour la 2ème semaine de mars 2026. Sur une durée totale de 90 heures la répartition est en accord avec la méthodologie Waterfall.



Graphique 1 Pourcentage des tâches planifiées

Ce premier graphique montre un équilibre entre les phases d'analyse, de conception et de test afin d'obtenir une organisation générale du projet. Les tâches liées à la documentation, qui accompagne le déroulement du projet, sont celles qui prennent le plus de temps. Et pour être réaliste, c'est la partie réalisation qui devrait prendre le plus de temps, car il s'agit d'un projet partant de zéro qui nécessite une technologie frontend et backend fonctionnelle.

2. Concepts

Partant des informations fournies par le cahier des charges, le projet est divisé en trois axes d'analyse principaux : l'analyse fonctionnelle liée au développement technique, les concepts UX/UI pour le frontend et la modélisation de la base de données.

2.1. Analyse Fonctionnelle

Les spécifications du cahier des charges permettent d'identifier les besoins fondamentaux du projet et de définir les interactions entre les utilisateurs et l'application.

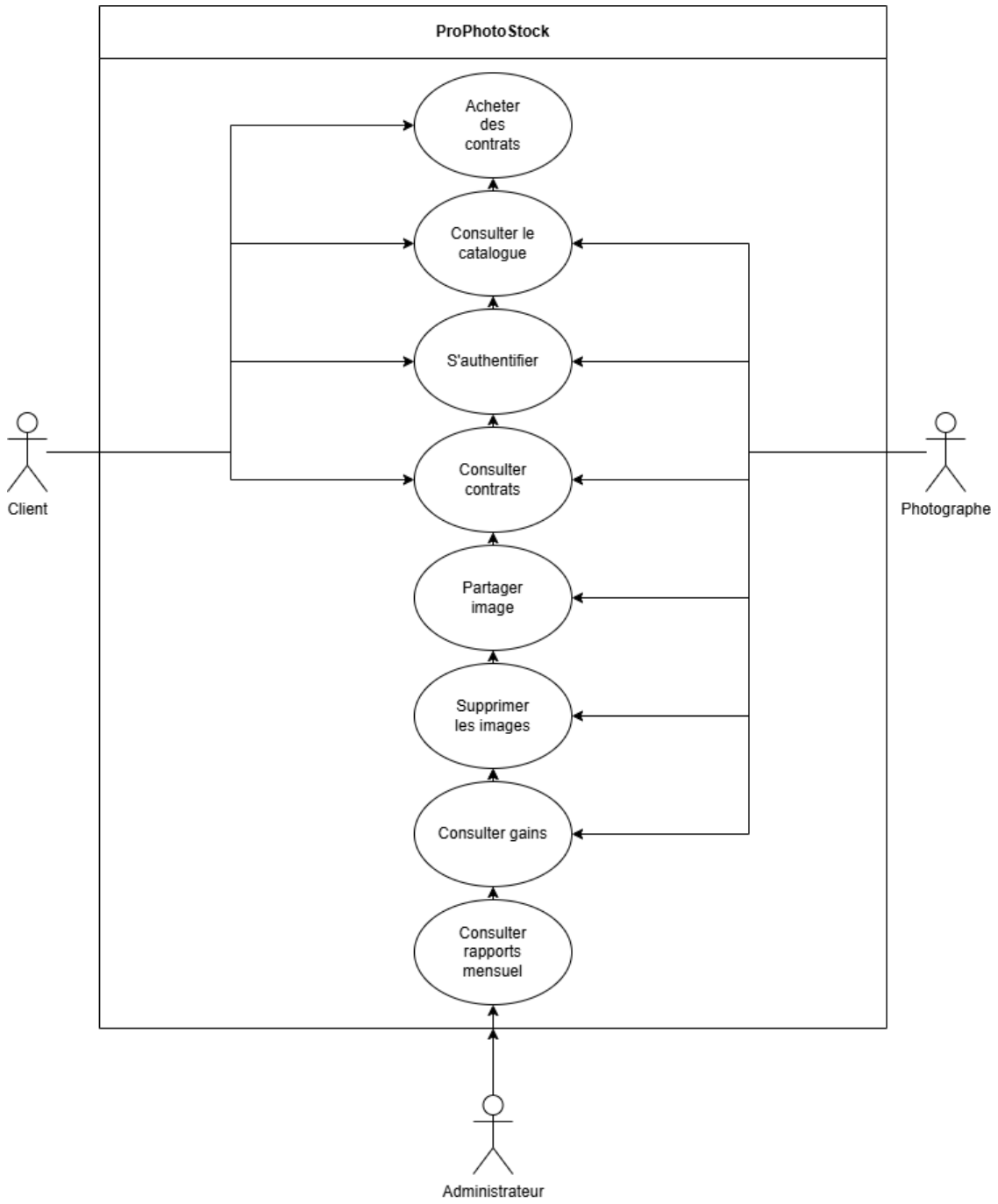


Schéma 3 Cas d'utilisation

Le diagramme de cas d'utilisation permet de modéliser les interactions entre les utilisateurs et les fonctionnalités de ProPhotoStock. Dans ce cas, tous les processus débutent par l'authentification afin d'identifier les rôles et les accès de chacun. Ensuite, tous les utilisateurs peuvent consulter le catalogue de photos, mais seul le client peut effectuer l'achat d'un contrat.

Du côté des photographes, ils ont la possibilité de prendre des photos et de les publier avec des tags et un titre. La fonction consulter les contrats est partagée entre tous, car les clients et les photographes peuvent voir leurs propres contrats, tandis que les administrateurs peuvent visualiser l'ensemble des gains. Enfin, les photos indésirables peuvent être supprimées soit par le photographe, soit par l'administrateur.".

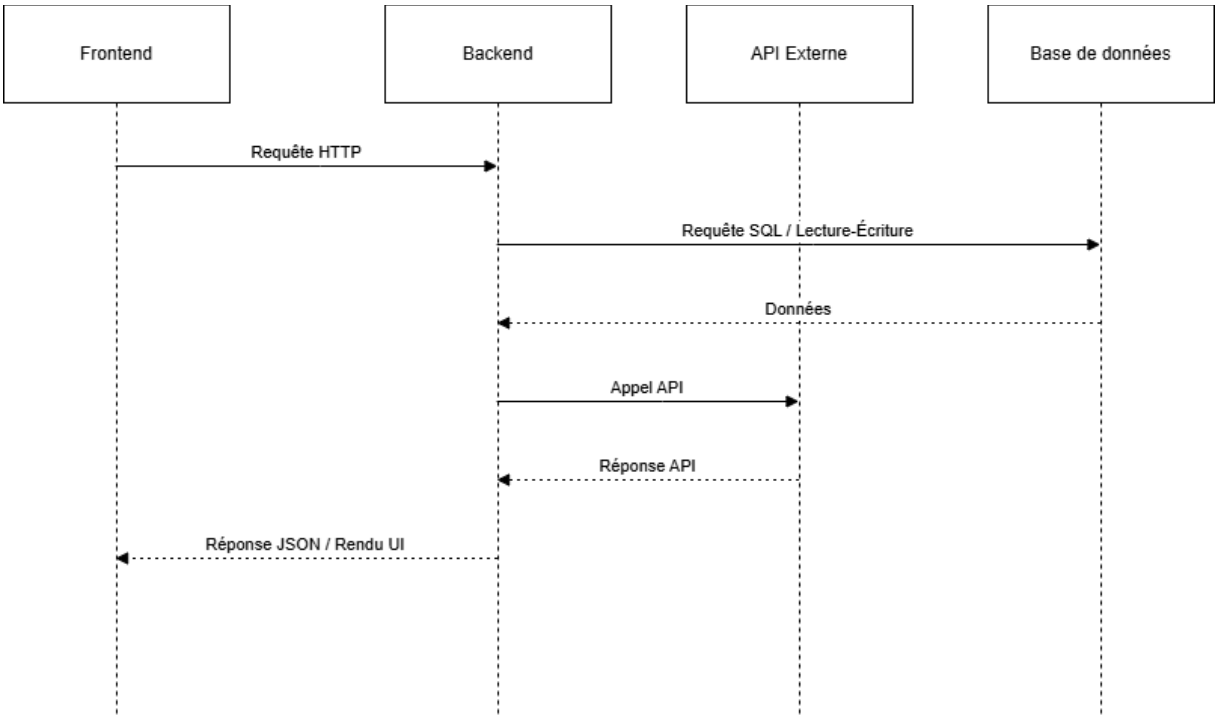


Schéma 4 L'architecture logicielle

L'architecture logicielle utilise un modèle Client-Serveur. Tout commence par le Frontend (MAUI) qui envoie une requête HTTP vers le Backend (Node.js). Le serveur traite la demande en effectuant des requêtes SQL auprès de la Base de données pour gérer les informations, et communique avec une API pour le stockage. Une fois les données récupérées, le Backend renvoie une réponse JSON au mobile pour assurer le rendu de l'interface utilisateur.

2.2. Flux de navigation

Le concept de ProPhotoStock App a l'architecture analysé sur l'architecture Client-Serveur et le diagramme de cas d'utilisation réalisés précédemment. Ces analyses permettent de connaitre les besoins fonctionnels et de définir la structure logique du système. De cette façon, le diagramme de flux suivant a été élaboré pour représenter la navigation que l'utilisateur suivra sur l'application mobile.

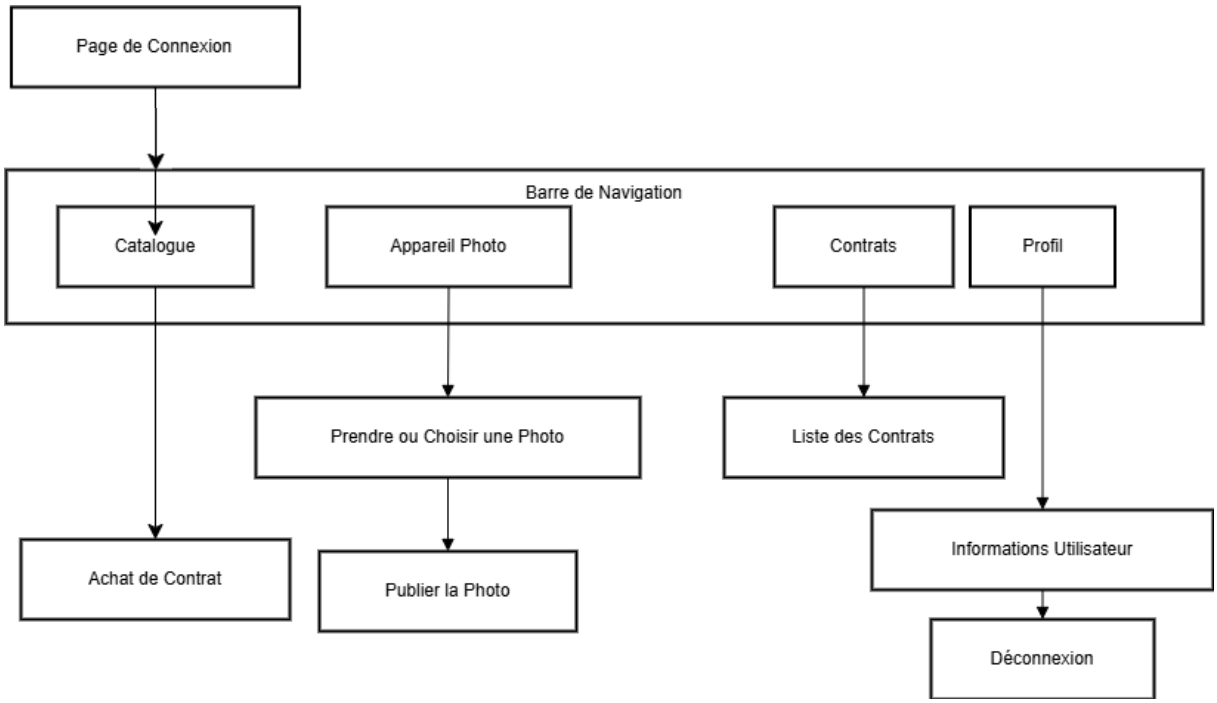


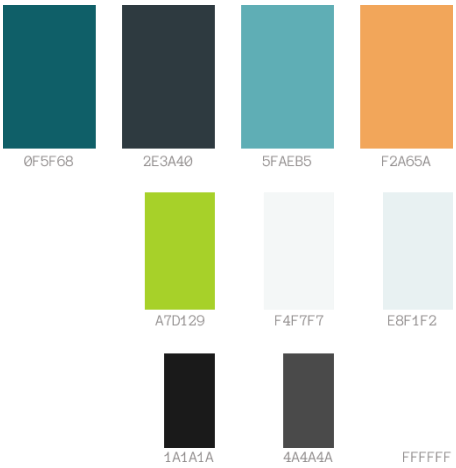
Schéma 5 Flux de navigation

Le flux de navigation de l'application est divisé en quatre sections principales, accessibles après une phase d'authentification. Cette première étape est indispensable pour identifier le profil de l'utilisateur et définir les fonctionnalités disponibles selon son rôle. Une fois connecté, l'utilisateur accède aux modules suivants :

- **Catalogue** (Accueil) : Cette section présente les photographies disponibles. Si l'utilisateur possède un profil "Client", il peut initier le processus d'achat de contrat directement.
- **Appareil Photo** : Dédiée principalement aux photographes, cette page permet de prendre ou choisir une photo depuis la galerie du smartphone afin de la publier avec ses informations associées.
- **Contrats** : Cet espace centralise la gestion des contrats numériques. L'accès aux données est filtré dynamiquement : chaque utilisateur ne visualise que les contrats liés à son activité et profil.
- **Profil** : Cette section permet de consulter les informations d'utilisateur et la possibilité de se déconnecter."

2.3. Maquettes UI/UX

- **Palettes de couleurs** : le développement des maquettes doit être réalisé à l'aide d'une palette de couleurs préalablement choisie et étudiée. En partant de la couleur dominante # 0F5F68, un vert foncé qui rappelle les camouflages utilisés par les photographes professionnels sauvages, les autres couleurs ont été extraites en tenant compte de leurs contrastes et de leur combinaison.

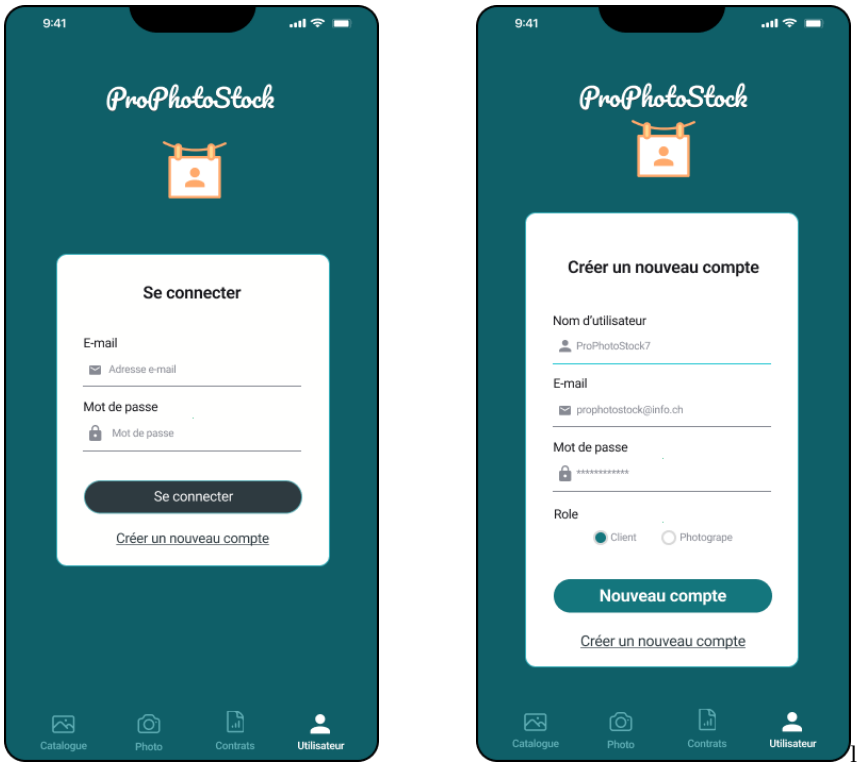


Maquette 1 Couleurs

- **Logo** : Ce design contient le nom de l'application en police Pacifico, suivi d'une icône représentant une photo imprimée suspendue à un câble courbé comme un sourire. Il s'inspire d'applications telles qu'Amazon et Instagram, qui combinent l'utilisation d'images et la vente.

Maquette 2 Logo

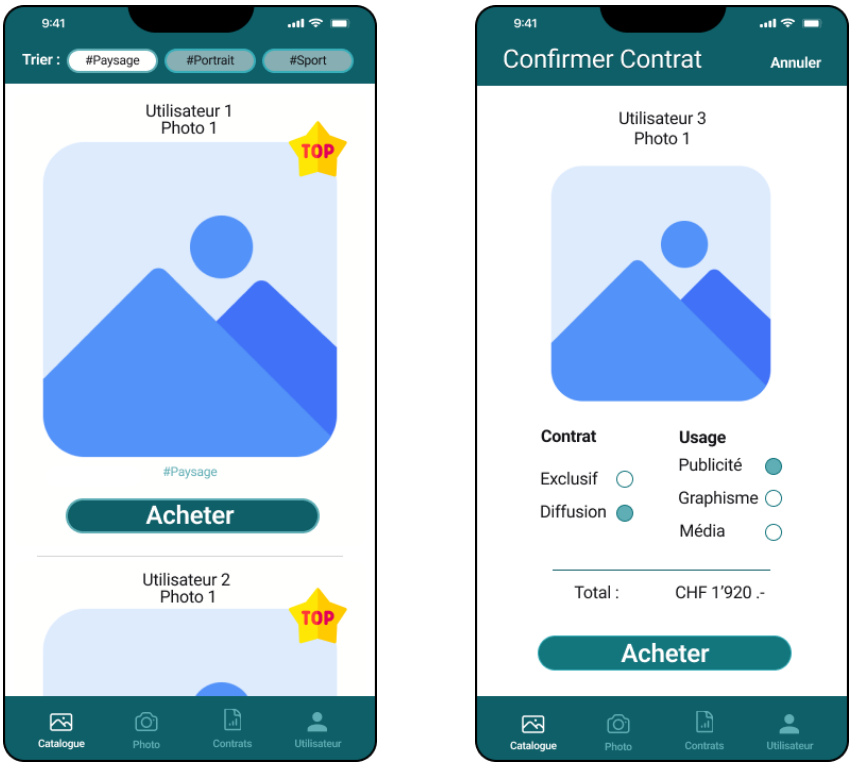
- **Authentification** : La première fois que l'utilisateur ouvre l'application, s'affiche directement à la page de connexion. L'adresse électronique et un mot de passe sont nécessaires pour l'accès. Et si l'utilisateur n'a pas un compte, il peut remplir le formulaire de création de compte avec son courriel, nom d'utilisateur et son rôle. En cas d'erreur, un message s'affiche pour en expliquer la raison



Maquettes 3 Authentification

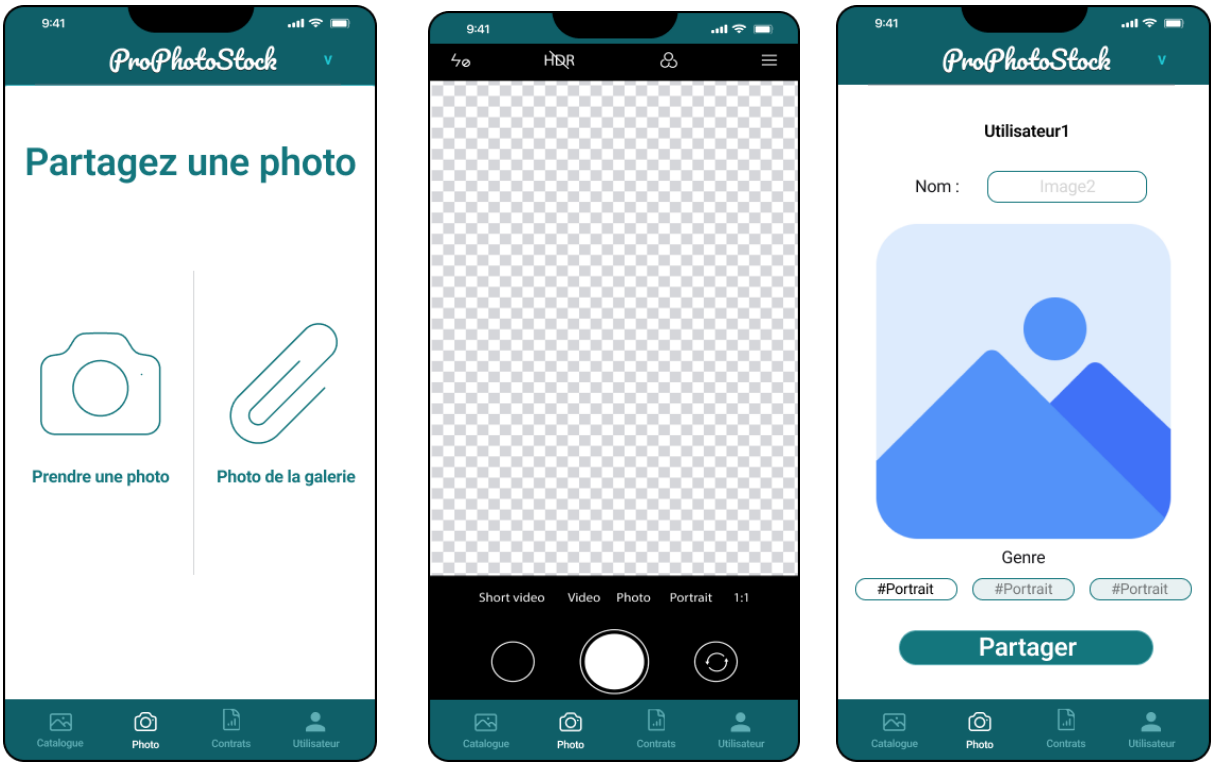
- **Catalogue :** Une fois connecté, la première page qui s'affiche est le catalogue d'images. Cette page à scroll vertical affiche toutes les images actives publiées par les photographes avec un titre, le nom de l'artiste et les catégories. Pour les clients, le bouton d'achat s'affiche afin de procéder à la demande d'un contrat. Les publications sont classées par popularité, de manière que l'image ayant fait plus de contrats s'affiche en premier. Et, au choix de l'utilisateur, trois boutons à sélectionner permettant d'effectuer une recherche par tags en haut de la page.

- **Confirmer contrat :** Cette page est réservée aux clients. Après avoir sélectionné une photo, les clients peuvent remplir le formulaire de contrat et procéder à l'achat. Pour ce formulaire, il est nécessaire de choisir le contrat souhaité et le mode d'utilisation, ce qui donne le prix total du contrat.



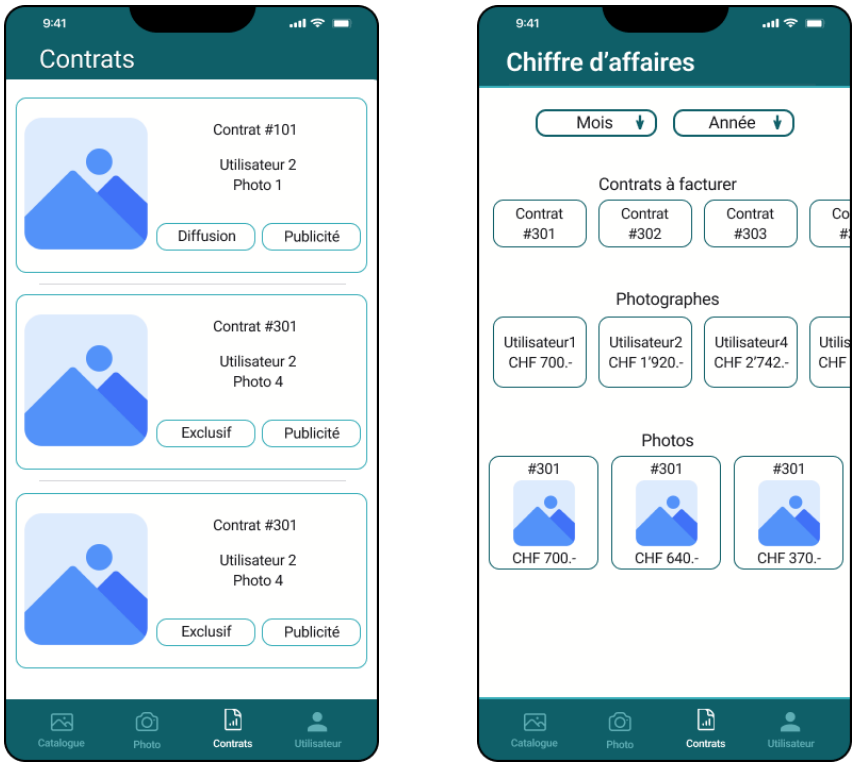
Maquette 4 Catalogue et Confirmer Contrat

- **Partage de photo :** Cette page est réservée aux photographes. Ils ont ici la possibilité de partager leurs images enregistrées ou prises à l'instant avec l'appareil photo de leur téléphone portable. Il leur suffit ensuite de choisir un titre pour enregistrer l'image et, s'ils le souhaitent, d'ajouter quelques tags appropriés.



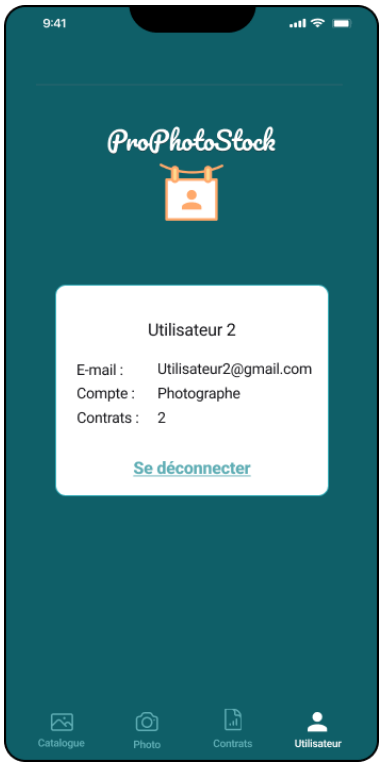
Maquette 5 Publication d'image

- **Contrats :** pour les client et photographes, chaque contrat actif ou expiré sera affiché avec toutes les informations pertinentes : numéro de contrat, image, auteur, titre, type de contrat et dépenses ou gains générés. Pour les administrateurs, la page des contrats correspond au chiffre d'affaires mensuel de l'entreprise. Vous pouvez y consulter toutes les informations relatives aux contrats facturés en sélectionnant le mois, les photographes et les photos publiées.



Maquette 6 Contrats et Chiffre d'affaires

- **Utilisateur :** enfin, cette page contient toutes les informations relatives à l'utilisateur. Comme son nom d'utilisateur, son adresse e-mail d'accès, son rôle dans le compte et le compteur des contrats correspondants. De plus, il est possible de se déconnecter de son compte.



Maquette 7 Profil

2.4. Modélisation des Données

La modélisation des données permet de structurer les informations de l'application afin de garantir l'intégrité des relations entre les utilisateurs, les photos et les contrats de vente.

- Modèle Conceptuel de Données (MCD)

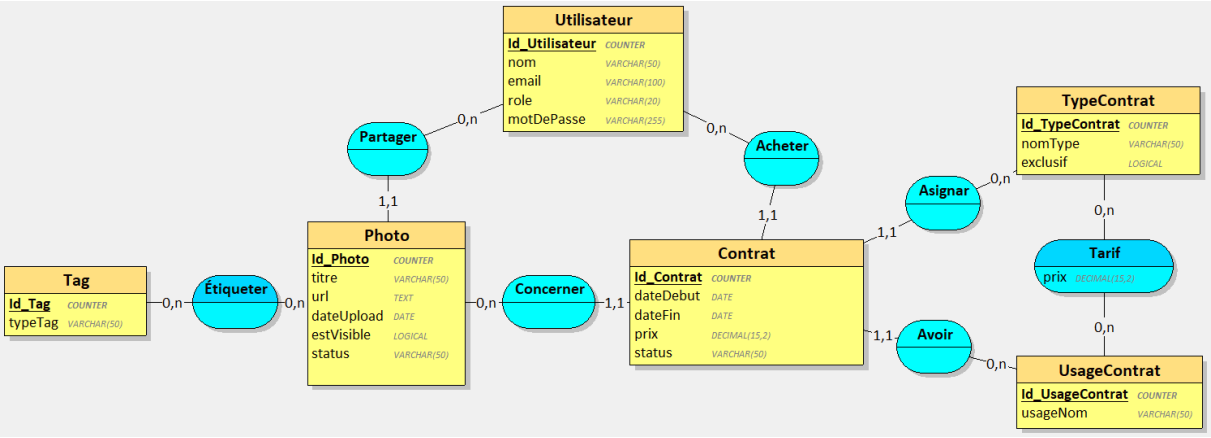


Schéma 6 MCD

- Gestion des Utilisateurs : L'entité Utilisateur centralise les informations de connexion (email, mot de passe) et définit le rôle qui conditionne les accès dans l'application.
- Publication de Photos : Un photographe (Utilisateur) peut Partager plusieurs photos, mais chaque photo appartient à un seul créateur. Les photos sont caractérisées par un titre, une url et un statut de visibilité.
- Contrats : le tableau Contrat lie un Client à une Photo via la relation Concerner. Chaque contrat enregistre la dateDebut, la dateFin et le prix appliqué selon les préférences du TypeContrat et UsageContrat.
- Les photos peuvent être Étiquetées par plusieurs Tag pour faciliter la recherche.

- Modèle Logique de Données (MLD)

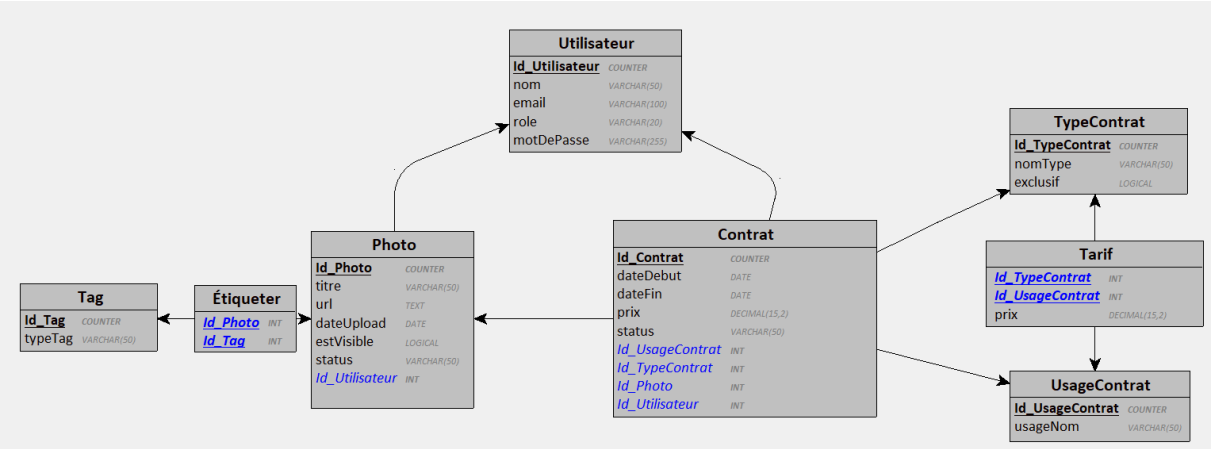


Schéma 7 MLD

Le MLD représente la structure finale des tables telles qu'elles sont implémentées dans la base de données MySQL avec ces clés primaires/étrangères et des associations.

2.5. Stratégie de test

La stratégie de test aide à garantir la fiabilité de différents actions et processus de l'application comme le transfert des images, la sécurité des accès et la stabilité de la base de données MySQL.

Types de tests :

- Tests Unitaires : valider méthodes y routes node.js
- Tests d'Intégration : valider la communication entre le Frontend MAUI et l'API.

Pour développer ces tests des outils comment la console de débogage de Test Unitaire de Visual Studio, Thunder sur Visual Studio pour requetés de l'API et son intégration et fonctionnement sur une émulateur Android Pixel 7. Des données fictives seront utilisées pour la création de comptes et de contrats de test, ainsi que des données réelles pour le partage de photographies. Une fois le plan de test établi, celui-ci sera soumis à la validation de ma cheffe de projet, puis vérifié directement dans l'application pour en consigner les résultats.

2.6. Risques techniques

Risques techniques	Solutions proposées
Manque de pratique récente avec MAUI et le développement backend	Revoir la théorie et projets précédents. Utiliser documentation et tutoriels
Complexité de la mise en place de l'environnement	Tester chaque composant avant l'intégration. Suivi mes guides du projets similaires précédents
Imprévus et incohérences d'intégration (routes, base de données, API, code…)	Tester les routes et des test unitaires. Une schéma SQL respectable. Logs pour identifier des erreurs
Gestion des médias	Système inconnu de ma part, à apprendre avec projets similaires sur Google ou IA et documentation
Manque de références et alors, risque de faire des choix sous-optimaux au début du projet.	Organiser des mini-meetings avec la cheffe d'équipe pour valider les choix.

2.7. Planification

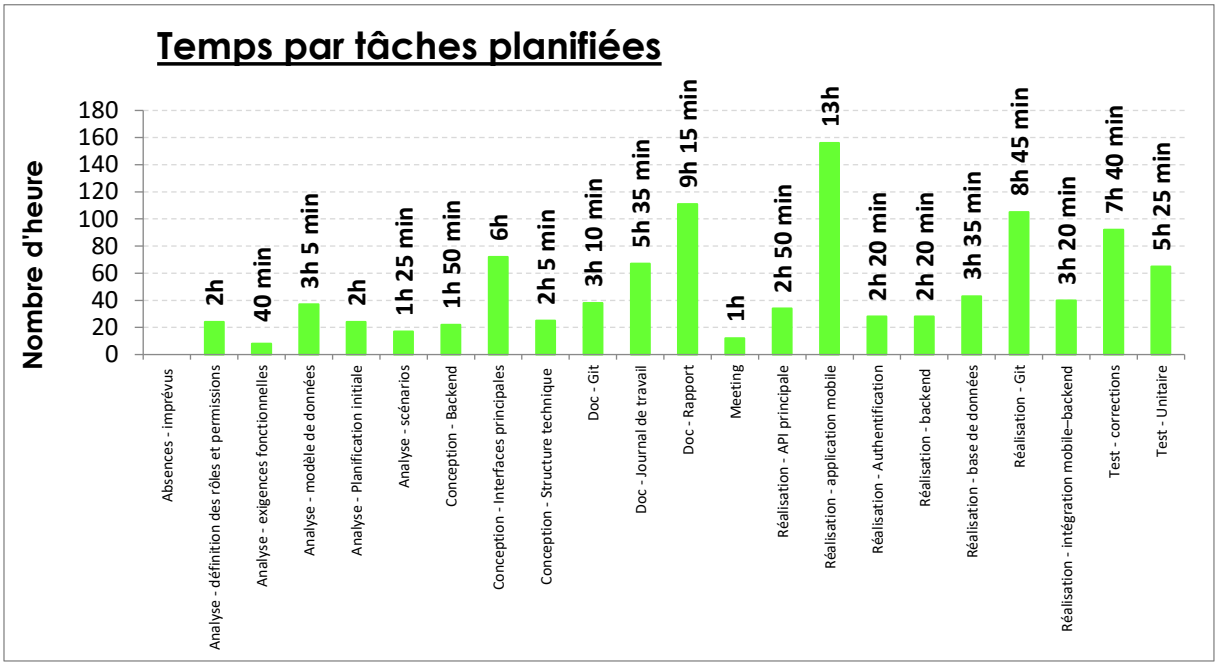
À la suite de l'analyse détaillée du concept et de l'identification des risques techniques, la planification initiale a été revue pour garantir le respect du délai imparti de 90 heures.

- **Dates du projet** : Le projet se déroule sur une période de 5 semaines, débutant le jeudi 5 février 2026 et se terminant le vendredi 13 mars 2026.
- **Ajustements de tâches** : Bien que la structure base reste fidèle au modèle Waterfall, j'ai décidé de donner temps dans la Réalisation (représentant 41.9%

du temps total soit environ 36 heures) pour pallier les imprévus liés à l'intégration de l'API Node.js avec MAUI et Docker.

- **Phases clés** : Conception & Analyse -> Réalisation -> Tests & Corrections Et en parallèle la Documentation.

Le tableau suivant présente une liste complète des tâches et le temps nécessaire à leur réalisation :



Graphique 2 Tâches planifiées

2.8. Dossier de conception

- **Choix du matériel (Hardware) et systèmes d'exploitation**
  - Environnement de développement : Un PC sous Windows 11 avec un processeur Intel 11th Gen i7 et 16 Go de RAM.
  - Environnement d'utilisation : L'application est conçue pour des smartphones Android 11.0 ou Superior. Testé avec émulateur de l'ETML Google Pixel 7.
  - Infrastructure Serveur : Un serveur local hébergeant des conteneurs Docker pour l'API et la base de données.
- **Outils logiciels :**
  - Frontend : Visual Studio 2022 avec le SDK .NET MAUI.
  - Backend : Visual Studio Code pour le développement de l'API Node.js et Thunder Client pour les tests de requête
  - Gestion de base de données : MySQL et Docker Desktop.
  - Services tiers : Git et Cloudinary.
- **Base de données**
  - **Modèle Relationnel (MLD et MCD)** : Le système repose sur un modèle relationnel structuré pour garantir l'intégrité des contrats. Les tables principales sont Utilisateurs, Photos, Contrats et Tags.
  - **Requêtes clés** : pour le bon fonctionnement des requêtes doit être formulaire pour les suivent fonctionnalités vitales : l'authentification,

l'affichage des images, créer des contrats, montrer mes contrats et information du profil.

- **Application mobile :** la conception de frontend est dessinée avec l'outil Figma, qui permet nous réaliser sur une maquette de vrai smartphone, une conception haute qualitéde notre interface.
- **Architecture du programme**

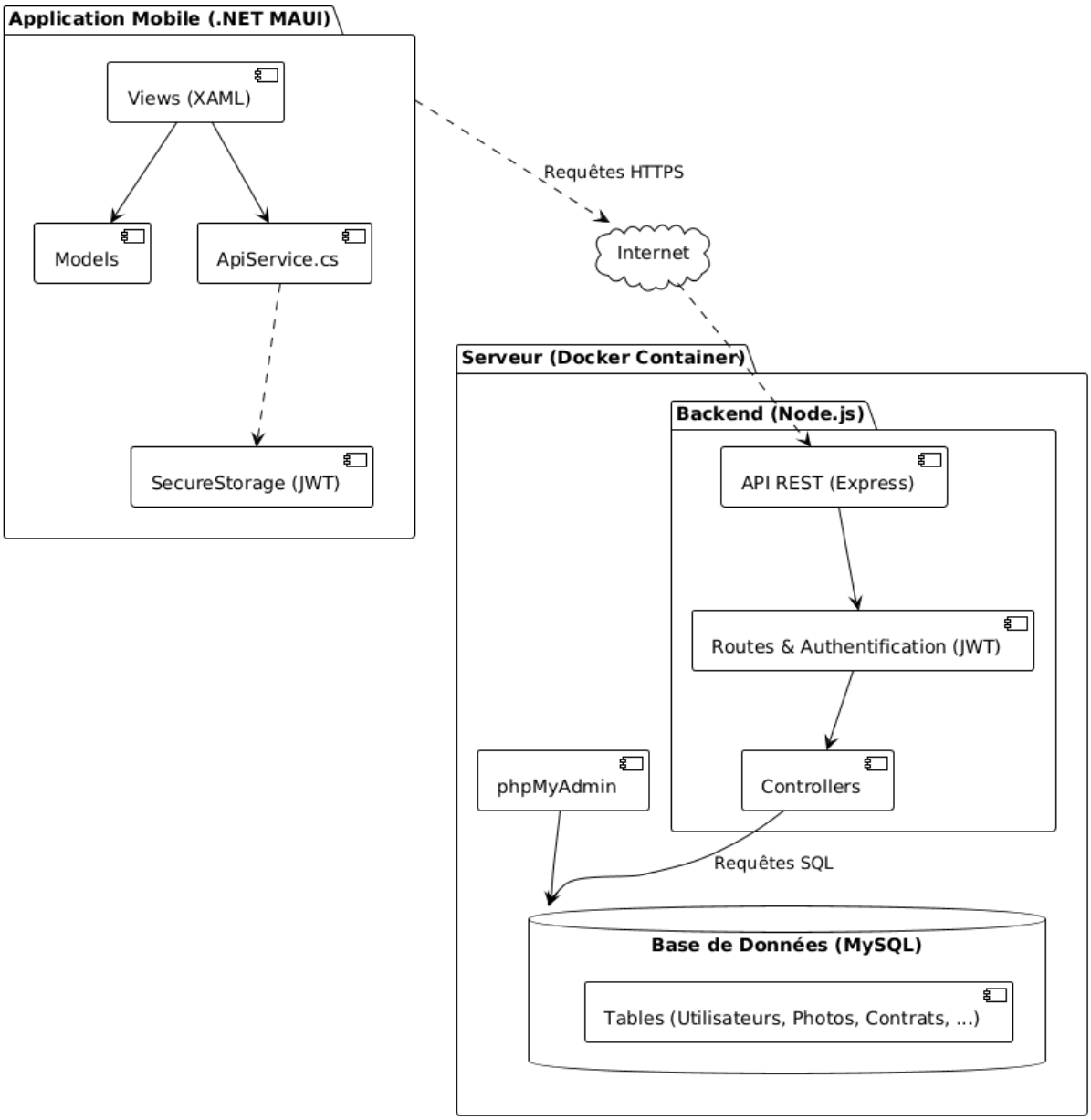


Schéma 8 Architecture Mobile

- **Logique du code** : La logique de mon code est conçue pour être modulaire MVC et sécurisée. L'organisation du le code est construit par la séparation entre les services de données (ApiService.cs), les modèles d'objets (Models) et les interfaces utilisateur (Views). Le Frontend MAUI s'occupe de l'expérience utilisateur et de la conversion des médias, le Backend Node.js garantit que toutes les transactions d'achats et publications respectent les règles définies dans mon modèle de données relationnel."

3. Dossier de réalisation

Le projet P_APPRO1_ProPhotoStock a été centralisé à l'aide de Git et GitHub Desktop. Ce gestionnaire de versions a permis de structurer le dépôt principal, lequel contient deux répertoires distincts : le dossier /app pour la solution frontend en .NET MAUI, et le dossier /backend pour la partie serveur, incluant : l'API Node.js, la base de données et la configuration Docker. Le projet a été régulièrement mis à jour par de *commits*, décrivant précisément chaque modification apportée au code source.

3.1. Arborescence et fichiers principaux

- *P_APPRO1_ProPhotoStock/app :*

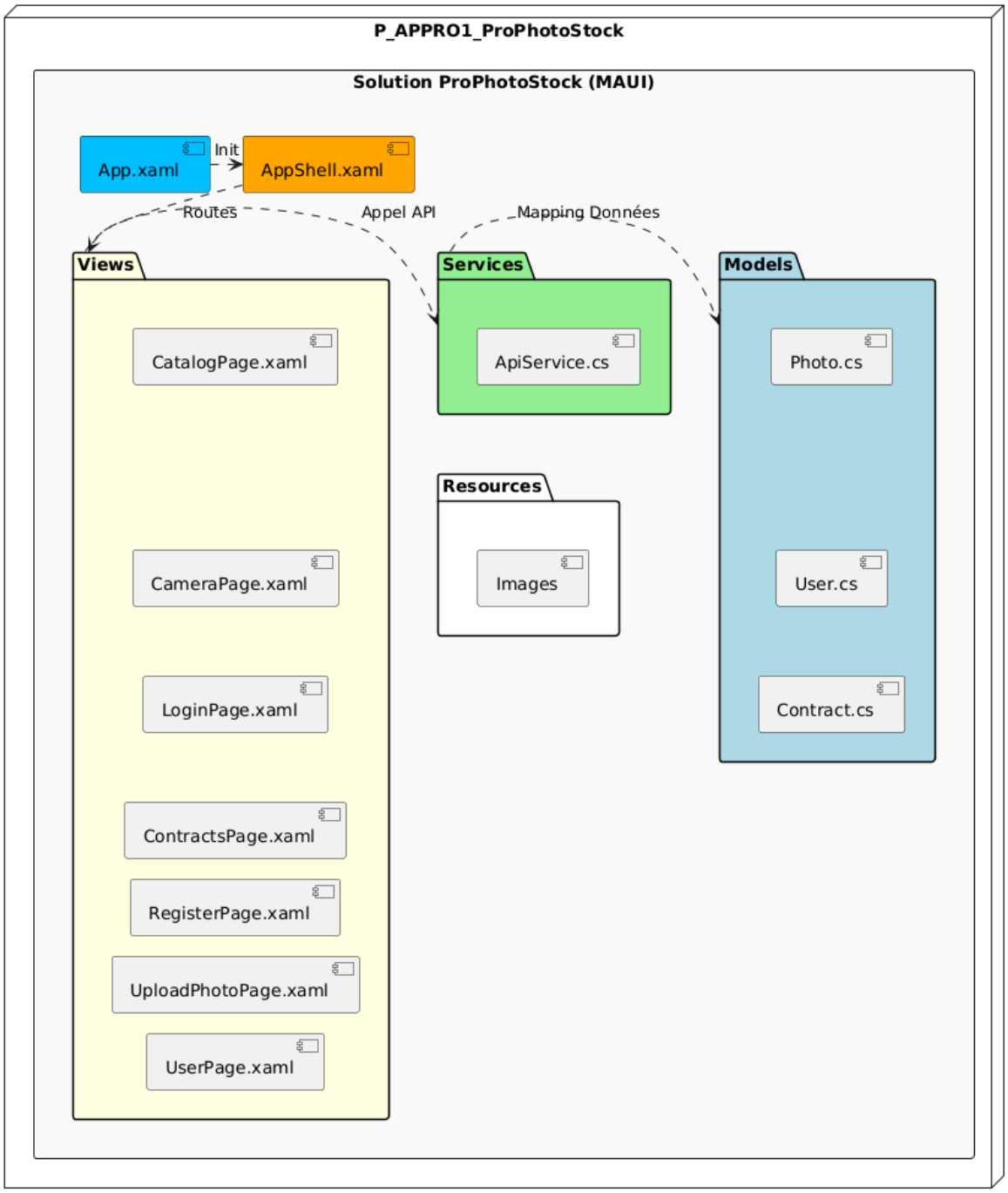


Schéma 9 Arborescence Frontend

Ce diagramme illustre l'architecture de la solution Frontend de l'application mobile. La logique est gestionné par App et AppShell, qui constituent le cœur de l'application et assurent la gestion de la navigation. L'interaction avec l'utilisateur se fait à travers le dossier Views, qui contient les interfaces des différentes pages. Le fichier ApiService est responsable de la gestion des requêtes HTTP vers le backend. De son côté, le dossier Models définit la structure des données reçues en réponse de l'API. Enfin, le répertoire Resources regroupe tous les éléments statiques tels que les images, les logos et les dictionnaires de styles utilisé.

- *P_APPRO1_ProPhotoStock/backend :*

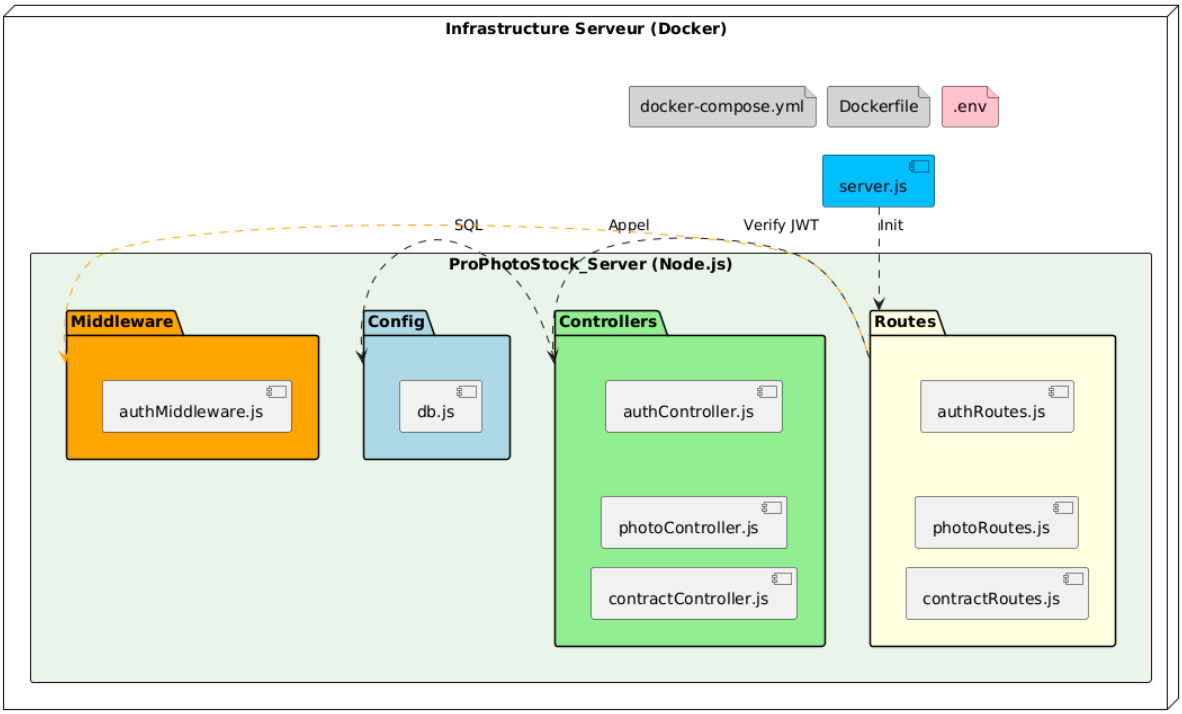


Schéma 10 Arborescence Backend

Pour comprendre le Backend, ce diagramme résume les codes et ressources utilisés. L'infrastructure repose sur l'intégration de fichiers Docker qui construisent l'environnement API et MySQL, complétés par un fichier « .env » pour sécuriser les variables sensibles. Le point d'entrée principal entre le serveur Express et les routes est le fichier server.js. Ce dernier gère les requêtes vers les différents endpoints de l'API (authentification, images, contrats). Ces routes sont sécurisées par un authMiddleware qui valide les Tokens JWT. Une fois la requête autorisée, la logique métier est géré par les contrôleurs, qui traitent les données et interagissent avec la base de données MySQL via le fichier de configuration db.js.

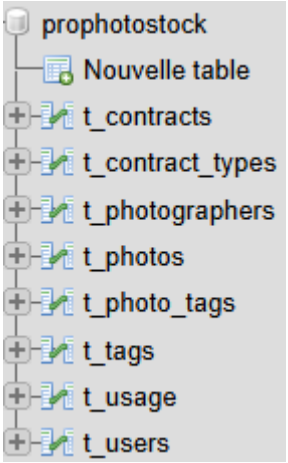


Schéma 11 Tables MySQL

NAME	IMAGE	STATUS	PORT(S)
backend 3 containers	-	Running (3/3)	-
nodejs-1 c64498ea9f86	backend_nodejs	Running	8080
db_container f7bd70408064	mysql	Running	6033
pma_container 205c62a74363	phpmyadmin	Running	8081

Figure 1 Container Docker

- *P_APPRO1_ProPhotoStock/doc :*

Enfin, toute la documentation accompagnant ce projet se trouve dans le dossier /doc, structuré en cinq parties principales : le cahier des charges, les maquettes de l'interface (UI/UX), le rapport final, la documentation technique de la base de données, et le journal de travail assurant le suivi chronologique du projet.

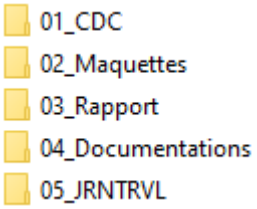


Figure 2 Dossier Doc

3.2. Versions des systèmes et outils

Composant	Outil	Version
Système d'exploitation	Windows 11 et Android 13	
Framework Frontend	.NET Maui	9.0
Environnement dév.	Visual Studio Community 2022	17.8
Runtime Backend	Node.js	20.11.0
Base de données	MySQL	8.0
Gestionnaire de paquets	npm	10.2.4
Conteneurisation	Docker Desktop	4.27.0

3.3. Description des tests effectués

La phase de validation et corrections du projet ProPhotoStock repose sur deux niveaux. D'abord, des tests unitaires ont été réalisés sur la partie mobile (MAUI) au sein d'un environnement xUnit Test Project. L'objectif principal est de valider la logique interne, comme le calcul des prix ou la gestion des droits d'accès par rôle. Parallèlement, le Backend a été éprouvée par des tests d'intégration à l'aide de l'outil Thunder Client. Ces tests ont pour but de vérifier le bon fonctionnement de l'API et la cohérence des échanges de données avec la base de données MySQL. Cette double validation assure que chaque composant logiciel répond aux exigences fonctionnelles du cahier des charges.

- Tests Unitaires

Type	Composant	Description	Résultat	Preuve
Unitaire	ContratsPage	Calcul du prix selon contrat et valeurs nuls.	Réussi	PriceCalculationTests (3) CalculatePrice_DiffusionMedia CalculatePrice_ExclusifPub CalculatePrice_NullValues
Unitaire	UserSession	Vérification des droits d'accès	Réussi	UserRoleTests (2) UserRole_Client_CannotUpload UserRole_Photographe_CanUpload
Unitaire	ApiService	Détection de Token JWT	Réussi	AuthServiceTests (2) Token_WhenNotEmpty_IsValid Token_WhenNull_IsInvalid
Unitaire	UploadPage	Validation du formulaire	Réussi	UploadTests (2) Upload_EmptyTitleOrFile Upload_ValidData

- Tests d'intégration

Endpoint	Description	Résultat	Preuve
POST /api/auth/login	Vérifier l'authentification et la génération du token JWT.	Code 200 OK et réception du token.	Status: 200 OK Size: 251 Bytes Time: 32 ms "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9 .eyJpZCI6NSwicm9sZSI6ImNsaWVudCIsImlhdCI6MT .YM_rBpJpGuJqKGAwGqC9ssoa3OUDxgbk-qCC5ly4Cy "user": { "username": "testclient", "role": "client", "email": "testclient@gmail.com"
GET /api/photos	Valider la récupération de la liste globale des photos.	Code 200 OK et un array de photos.	Status: 200 OK Size: 1.57 KB Time: 9 ms "photoId": 1, "photoTitle": "Test", "photoUrl": "https://encrypted-tbn0.gstatic.com -uJw&s", "useName": "testphoto", "tagName": "Paysage"
GET /api/photos/tag/Sport	Tester le filtrage par catégorie	Seules les photos avec le tag "Sport" sont renvoyées.	Status: 200 OK Size: 791 Bytes Time: 8 ms "photoId": 8, "photoTitle": "mauiTest2", "photoUrl": "https://res.cloudinary.com/dzguf69ws/ /o2dofb9suqn07qg8owqw.jpg", "useName": "testphoto", "tagName": "Sport"
POST /api/contracts	Valider la création d'un contrat	Code 201 Created et enregistrement en DB.	Status: 200 OK Size: 79 Bytes Time: 32 ms "message": "Contract completé", "details": { "total": "700.00", "exclusive": false

GET /api/contracts/my-contracts	Vérifier que l'utilisateur accède uniquement à ses propres achats.	Liste filtrée par l'ID de l'utilisateur connecté.	Status: 200 OK Size: 1.65 KB Time: 7 ms "contractId": 3, "startDate": "2026-02-26T14:48:52.000Z", "endDate": "2027-02-26T14:48:52.000Z", "price": "740.00", "status": "active", "photographerCommission": "148.00", "fkUsage": 1, "fkType": 1, "fkPhoto": 1, "fkUser": 5, "photoTitle": "Test", "photoUrl": "https://encrypted-tbn0.gstatic.co
POST /api/photos	Tester l'upload d'une nouvelle photo	Image stockée sur Cloudinary et entrée créée en DB.	Status: 200 OK Size: 154 Bytes Time: 1.88 s "message": "Photo post", "photoId": 11, "url": "https://res.cloudinary.com/dzguf69ws/

3.4. Erreurs restantes

Description du problème	Conséquences	Actions envisagées
Gestion du flux de travail : Le passage fréquent entre phases de développement, analyses et de conception a perturbé le planning initial.	Perte de temps et suivi du journal de travail irrégulier.	Adopter correctement la méthodologie Waterfall et finaliser les étapes correctement avant de continuer.
Conception UI/UX : Absence de guide de style au début du projet, entraînant des ajustements graphiques tardifs.	Incohérences visuelles mineures et retravail sur l'alignement des éléments.	Utiliser correctement Figma avec griller de mise en page et images de même styles.
Réutilisation de code : Adaptation de code provenant d'anciens projets pour le Backend.	Temps important passé à la comprenions et intégration du code, plutôt qu'à la création de nouvelles fonctionnalités.	Privilégier le développement pas à pas, plutôt que l'intégration sans connaissance.
Priorisation des tâches : Focus important sur les fonctions secondaires (Caméra, Cloudinary) au détriment du système de paiement et le tableau de gains.	Le système de paiement reste sans être mis en place.	Trier les tâches sur la planification Git par priorités.
Manque de restrictions : Les messages d'erreur et les restrictions de saisie (validation email/mot de passe) sont basiques ou inexistants.	Risque de données erronées et expérience utilisateur moins intuitive.	Implémenter des expressions et des DisplayAlert plus détaillés pour guider l'utilisateur.

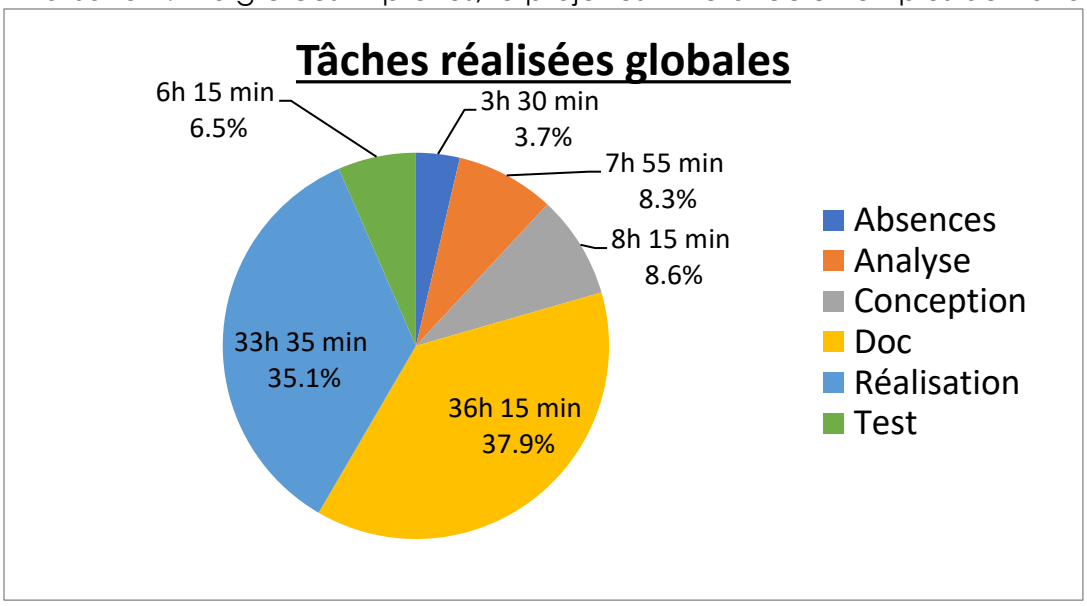
3.5. Liste des documents fournis

Document	Version	Format	Description
ProPhotoStock (MVP)	V1.0	Code Source	Solution MAUI complète et Serveur API Node.js.
Rapport	V1.0	PDF	Document technique
Journal de Travail	V1.0	Excel	Suivi quotidien de l'évolution du projet.
Manuel d'Installation	V1.0	PDF (Annexe)	Procédure de déploiement Docker et .NET.
Manuel d'Utilisation	V1.0	PDF (Annexe)	Guide utilisateur avec captures d'écran pour la navigation
Dépôt Git		URL	Accès à l'historique complet des commits. https://github.com/Adrtoledocam/P_APPRO1_ProPhotoStock

3.6. Analyses de temps réalisé

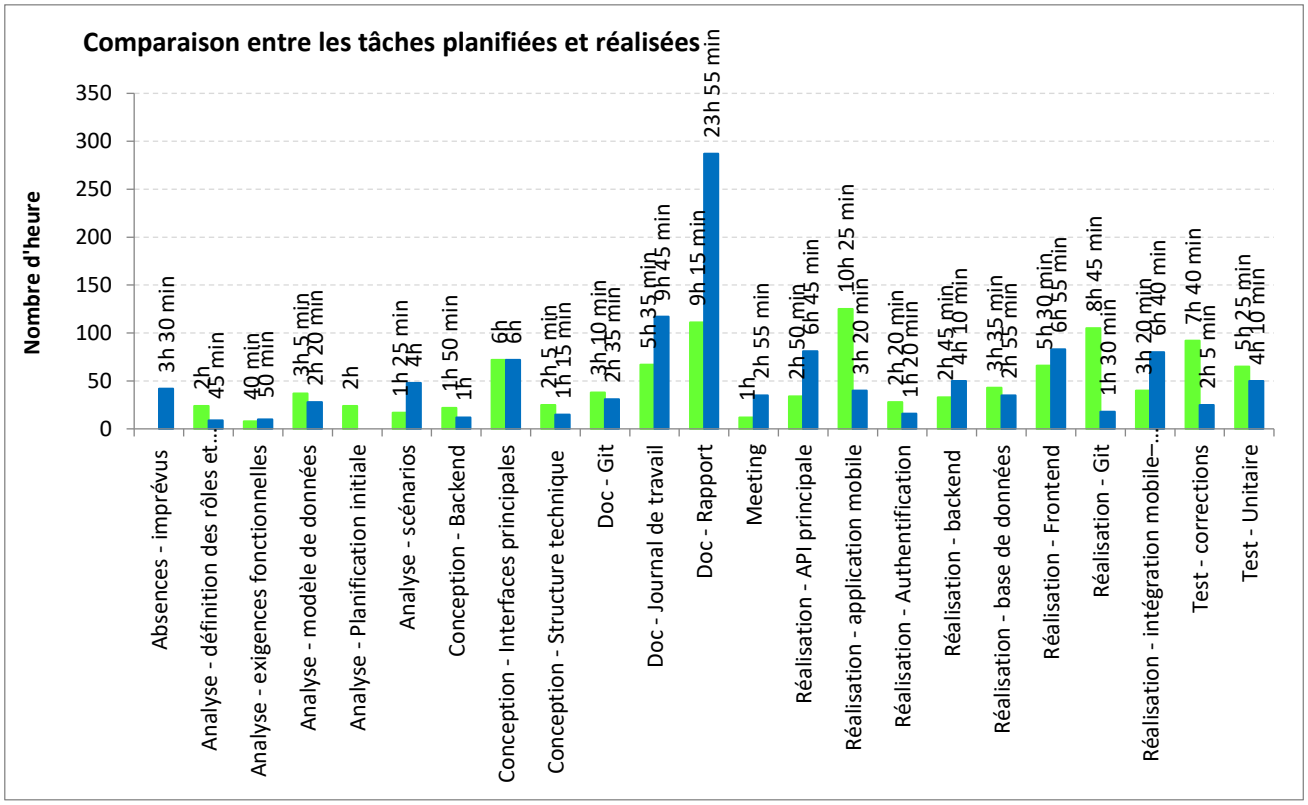
Cette section compare la planification initiale (Graphique 1 et 2) avec le travail réellement effectué durant les 90 heures du projet suit la méthodologie Waterfall prévue.

- Développement (Réalisation) : Bien que ce soit la phase la plus importante, elle a nécessité une attention particulière pour configurer la communication entre l'émulateur Android et les conteneurs Docker
- Conception et Analyse : Phases réalisée selon les prévisions, permettant de valider l'UI/UX et la base de données (MCD).
- Finalisation : Malgré ces imprévus, le projet est livré avec 8h en plus de travail.



Total heures planifiées 90h 40 min **Total heures réalisées** 98h 40 min **Différence** + 8h

Graphique 3 Tâches réalisées



Graphique 4 Temps par tâches réalisées

4. Conclusions

Après 90 heures de travail réparti sur 5 semaines, ce projet touche à sa fin avec la présentation du premier prototype de l'application ProPhotoStock. Ce « Minimum Viable Product » atteint les objectifs principaux définis dans le cahier des charges et répond aux attentes de ma responsable de projet.

4.1. Objectifs

- Objectifs atteints : La première version propose une architecture client-serveur opérationnelle incluant les fonctionnalités suivantes : authentification des comptes, consultation du catalogue, achat et gestion de contrats, publication de photos, consultation de profil, gestion des rôles, calcul dynamique des prix et intégration de Cloudinary pour le stockage d'images dans le cloud.
- Objectifs non atteints : Par manque de temps et par suite d'une confusion des priorités du cahier des charges, l'implémentation d'un système de paiement réel a été écartée de cette version. De plus, certaines optimisations concernant la sécurité avancée, la validation des formulaires et un système d'alertes utilisateur plus complet auraient pu renforcer ce projet.

4.2. Points positifs

- Architecture complète : Bien qu'il ne s'agisse pas de mon premier projet MAUI, c'est celui où j'ai réussi l'intégration la plus complète d'une architecture client-serveur. Cette réussite est très satisfaisante et m'a permis de comprendre l'importance d'anticiper la structure du backend dès la phase d'analyse et conception du frontend.
- Apprentissage de la technologie .NET MAUI : Le développement avec ce Framework a été un véritable défi. Sa diversité fonctionnelle pour le développement mobile m'a poussé à approfondir mes connaissances. Les nouvelles compétences acquises de code et de logique de développement seront améliorées pour mes futurs projets.

4.3. Points négatifs

- Gestion du temps : Une sous-estimation du temps nécessaire aux phases d'analyse, de conception et de développement a rendu le suivi du planning initial difficile. J'ai pris conscience qu'avec l'expérience, il est crucial de définir une planification plus réaliste. Pour ce projet, certains problèmes entre les prévisions et la réalité rappellent que la méthodologie Waterfall doit être suivie avec rigueur pour être efficace.
- UX/UI : Malgré ma satisfaction concernant l'interface, je reconnais des manques de professionnalisme sur certains principes de base : respect des règles d'alignement, simplification des parcours utilisateurs et amélioration globale de l'expérience UX. Ces retouches tardives ont provoqué des irrégularités dans mon journal de travail.

4.4. Difficultés particulières

Le défi majeur a été la mise en place du backend et la configuration de l'environnement de développement, parallèlement à l'implémentation des tests. L'intégration de Docker pour orchestrer une API, une base de données et assurer la communication avec les services .NET MAUI représentait une barrière technique importante. Cette étape m'a donné confiance et m'a appris l'importance d'une analyse plus étendue de l'architecture logicielle avant de commencer le codage.

4.5. Suites possibles pour le projet

Si je devais poursuivre le développement, ma priorité serait de finaliser le système de paiement pour les contrats, des compétences essentielles dans le développement mobile actuel. J'aimerais également renforcer la sécurité et la gestion des rôles pour personnaliser l'interface selon le profil utilisateur. Enfin, l'ajout de statistiques de vente, de gestion de factures et de notifications push (pour confirmer la signature d'un contrat ou annoncer les gains mensuels) constituerait une évolution majeure pour l'application mobil.

5. Annexes

5.1. Résumé du rapport du TPI

5.1.1. Contexte et mandat

Dans le cadre du projet P_APPRO1, le développement de ProPhotoStock a été entrepris. Cette application mobile est dédiée à la gestion et à la commercialisation de licences photographiques. Le mandat consistait à concevoir une solution technique permettant de mettre en relation des photographes professionnels et des clients. L'enjeu principal résidait dans l'implémentation d'un système capable de gérer divers types de contrats et de sécuriser le stockage des fichiers multimédias, tout en proposant une interface intuitive pour la consultation du catalogue.

5.1.2. Solution réalisée

La solution logicielle repose sur une architecture client-serveur moderne. Le frontend a été développé avec le framework .NET MAUI, optimisé pour un déploiement sur Android. Le backend est constitué d'une API REST bâtie avec Node.js et Express, conteneurisée via Docker. La persistance des données est assurée par une base de données MySQL, tandis que le service Cloudinary gère l'hébergement des images dans le cloud. La sécurité est garantie par l'utilisation de tokens JWT pour l'authentification et une gestion stricte des rôles utilisateur (Client, Photographe, Administrateur).

La gestion du code source a été centralisée sur un dépôt Git, permettant un suivi rigoureux des versions. De plus, un dossier documentaire complet accompagne le projet, incluant le journal de travail, le cahier des charges et les manuels techniques d'installation et d'utilisation. Cet ensemble garantit une base solide pour la maintenance et l'évolution future du logiciel.

5.1.3. Résultats principaux

Le projet a un prototype fonctionnel répondant aux exigences du cahier des charges. Les utilisateurs disposent des fonctionnalités d'inscription, de connexion, de filtrage des photos par catégories et de simulation d'achat avec un calcul dynamique des prix. Les photographes bénéficient de la fonction de partage d'images via la caméra ou la galerie de l'appareil. La fiabilité du code a été validée par une série de tests unitaires (xUnit) et de tests d'intégration (Thunder Client), sécurisant ainsi les processus de facturation et les accès aux données.

5.1.4. Bilan et suites

Si bien que les objectifs principaux de l'architecture et des fonctionnalités CRUD soient atteints, le projet présente des limites liées aux contraintes temporelles, notamment l'absence d'une passerelle de paiement réelle et d'un tableau de bord financier mensuel. Ce prototype constitue une base technique robuste. Les suites logiques incluent l'intégration de notifications Push, l'implémentation de statistiques de vente et une optimisation de l'UX/UI sur l'ergonomie et accessibilité pour une produit final.

5.2. Vocabulaire

Terme	Description
API REST	Interface de programmation permettant l'échange de données entre le frontend et le backend via des requêtes http.
Asynchrone (Aync)	Mode d'exécution permettant de ne pas bloquer l'application mobile pendant qu'une tâche
Client-Serveur	Communication où l'application mobile (Client) consomme des ressources mises à disposition par le serveur (API).
Cloudinary	Service cloud externe utilisé pour l'hébergement et la gestion des images du projet.
Commit	Action d'enregistrer une étape du développement dans l'historique Git.
Container et Docker	Permettant d'exécuter l'API et la base de données dans des environnements.
CRUD	Acronyme pour Create (Créer), Read (Lire), Update (Modifier), Delete (Supprimer).
Data Binding	Lien dynamique entre les données du code (C#) et l'interface utilisateur (XAML) dans .NET MAUI.
Dépôt (Repository)	Espace de stockage centralisé sur Git.
Endpoint	Adresse URL par l'API pour effectuer une action.
Framework (.NET MAUI)	Cadre de travail utilisé pour compiler de code vers Android.
Maquette UI	Représentation visuelle et statique de l'interface utilisateur.
Middleware	Couche logicielle intermédiaire chargée de filtrer les requêtes.
MVP (Minimum Viable Product)	Version simplifiée du produit
MySQL	Système de gestion de base de données relationnelle utilisé pour stocker les données du projet.
Requête HTTP	Demande envoyée par le client au serveur
Scalabilité	Capacité du système à s'adapter et fonctionner avec un grand nombre d'utilisateurs.
Tests d'Intégration	Tests pour valider la communication entre l'API.
Tests Unitaires (xUnit)	Procédure de vérification à tester fonctions.
Token JWT	Jeton de sécurité crypté.
XAML	Langage de balisage utilisé pour définir le design de l'interface mobile.

5.3. Manuel d'Installation

Ce manuel, et aussi sur le README du git, décrit les étapes nécessaires pour déployer l'environnement de développement et exécuter l'application ProPhotoStock.

a) Prérequis système

Avant de commencer, assurez-vous que les logiciels suivants sont installés sur votre machine :

- Docker Desktop (version 4.0 ou supérieure).
- Visual Studio 2022 avec la charge de travail .NET MAUI.
- Git (ou GitHub Desktop).
- Un émulateur Android (configuré via Visual Studio).

b) Récupération du projet

Clonez le dépôt distant contenant le code source :

```
git clone https://github.com/Adrtoledocam/P_APPRO1_ProPhotoStock.git
cd ProPhotoStock
```

c) Déploiement du Backend (Docker)

L'API et la base de données sont orchestrées via Docker. Pour les lancer :

1. Ouvrez un terminal à la racine du dossier Backend.
2. Exécutez la commande suivante :

```
docker-compose up -d --build
```

3. Vérifiez dans Docker Desktop que les conteneurs api_node et mysql_db sont en état "Running".

d) Configuration du Frontend

Pour que l'application mobile puisse contacter l'API fonctionnant sous Docker :

1. Ouvrez la solution .sln dans Visual Studio.
2. Accédez au fichier de configuration de l'API (ex: Config.cs o ApiService.cs).
3. Important : Si vous utilisez l'émulateur Android, remplacez localhost par 10.0.2.2 dans l'URL de l'API :

```
public static string BaseUrl = "http://10.0.2.2:3000/api/";
```

e) Lancement de l'application

Dans Visual Studio, sélectionnez l'émulateur Android comme cible de déploiement. Démarrer le débogage. L'application s'installe sur l'émulateur et la page de connexion s'affiche.

5.4. Manuel d'Utilisation

Ce manuel décrit les procédures opérationnelles pour les différents profils d'utilisateurs de l'application ProPhotoStock.

a) Gestion du compte

- Inscription : Sur l'écran de création de compte, l'utilisateur doit saisir ses informations personnelles (Nom, Email, Mot de passe) et sélectionner impérativement son rôle : "Client" ou "Photographe".
- Connexion : Une fois le compte créé, l'utilisateur s'identifie pour générer son token de session.
  Voir Annexe 5.6 Archives du projet : Figure 8 pour une illustration de l'interface

b) Guide du profil Client

Le client utilise l'application pour acquérir des droits sur des images.

1. Navigation : Parcourir la galerie dans l'onglet "Catalogue".
2. Sélection : Cliquer sur une photo « acheter » pour ouvrir les détails.
3. Achat de licence :
   - Sélectionner le type de contrat (Exclusif ou Diffusion).
   - Préciser le type d'usage.
   - Appuyer sur le bouton "Confirmer" pour valider la transaction.
4. Consultation : Les photos acquises sont listées dans l'onglet "Contrats".

Voir Annexe 5.6 Archives du projet : Figure 5, 6 et 7 pour une illustration de l'interface

c) Guide du profil Photographe

Le photographe utilise l'application pour alimenter le catalogue et suivre ses ventes.

1. Partage d'image : Dans l'onglet d'upload, le photographe peut utiliser la caméra pour une prise de vue directe ou sélectionner un fichier dans la galerie du téléphone.
2. Publication : Après avoir défini le titre et la catégorie, cliquer sur "Partager" pour envoyer l'image vers Cloudinary et l'API.
3. Suivi : Consulter l'onglet "Contrats" pour voir l'historique des gains.

Voir Annexe 5.6 Archives du projet : Figure 3 et 4 pour une illustration de l'interface

d) Guide du profil Administrateur

L'interface de l'administrateur est identique à celle des autres utilisateurs pour garantir une navigation simple, avec une modification spécifique :

- Contrats : Dans la section normalement dédiée aux contrats, l'administrateur accède à la vue "Rapport Mensuel". Cette section permet de superviser l'activité globale de la plateforme sur le mois en cours.

Voir Annexe 5.6 Archives du projet : Figure 10 pour une illustration de l'interface

5.5. Archives du projet

- Captures d'écrans du ProPhotoStock App :

Figure 8 Se connecter

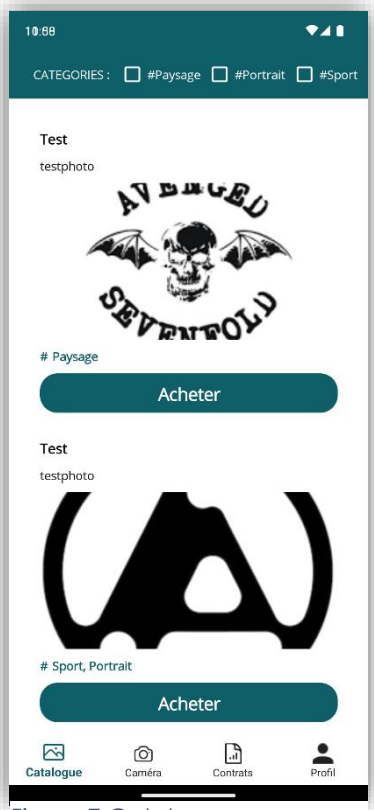


Figure 7 Catalogue

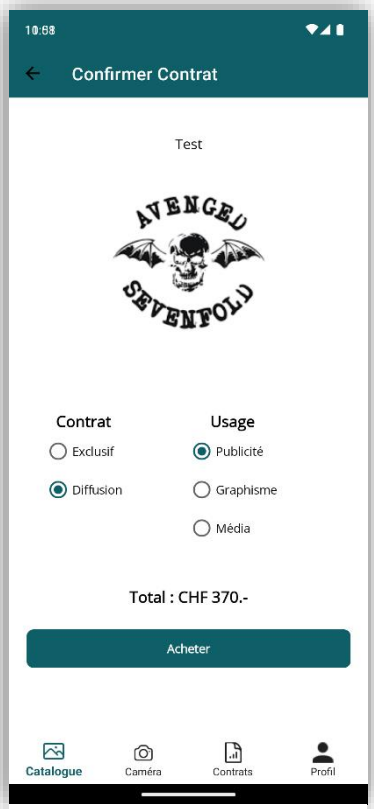


Figure 6 Confirmer Contrat

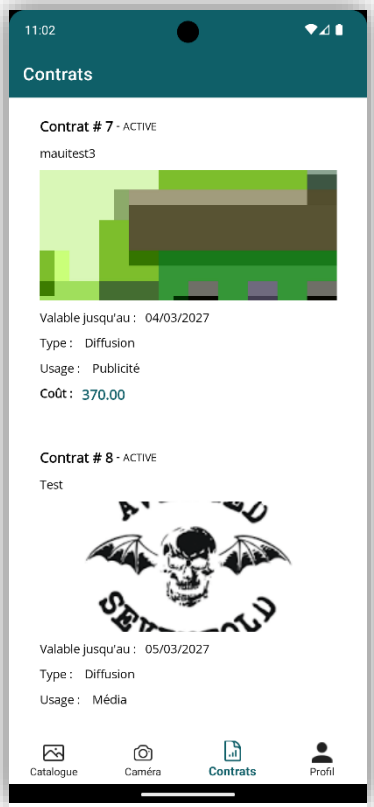


Figure 5 Contrats

Figure 4 Partager

Figure 3 Formulaire de publication

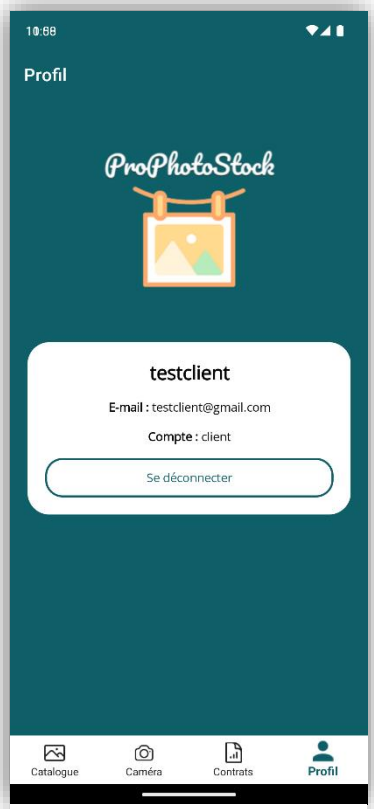


Figure 10 Profil

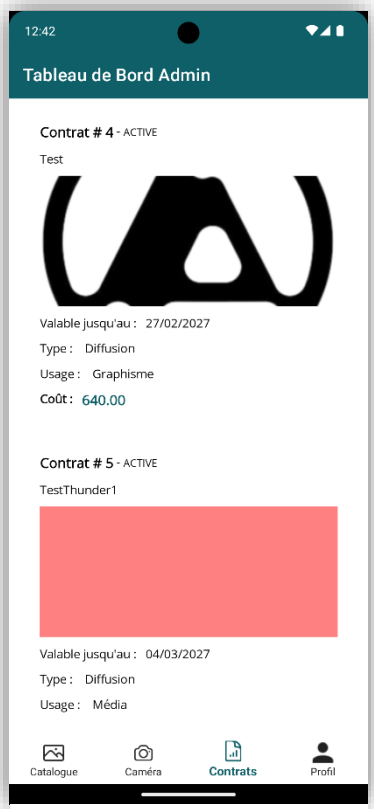


Figure 9 Tableau de Bord

5.6. Sources – Bibliographie

- **Documentation officielle**
  - Microsoft. (s. d.). *Microsoft Learn – .NET MAUI documentation*. https://learn.microsoft.com/fr-fr/dotnet/maui/
  - Express.js. (s. d.). *Express.js – Node.js web framework*. https://expressjs.com/fr/
  - Cloudinary. (s. d.). *Cloudinary – Image management API*. https://cloudinary.com/documentation
  - Docker. (s. d.). *Docker documentation*. https://docs.docker.com/
  - Microsoft. (s. d.). *MediaPicker for .NET MAUI*. https://learn.microsoft.com/fr-fr/dotnet/maui/platform-integration/device-media/picker
- **Forums et ressources techniques**
  - Stack Overflow. (2018). *How to access localhost from Android emulator*. https://stackoverflow.com/questions/50389325/how-to-access-localhost-from-android-emulator
- **Vidéos YouTube Espagnol**
  - Microsoft Developer. (2022). * .NET MAUI para principiantes* [Vidéo]. YouTube. https://www.youtube.com/watch?v=ZUsD6KRKz0w
  - Luis Beltrán. (2023). *Tomar fotos y obtener de galería en .NET MAUI* [Vidéo]. YouTube. https://www.youtube.com/watch?v=llcxG4m13DY
- **Projets utilisés**

- Innoweek – Feelings 2024.
- MAUI Flashcards.
- MAUI Library.
- API REST – 14 steps.

- **Professeurs consultés**
  - Meylan, M. *Retour sur l'expérience et l'interface utilisateur (UX)*. Communication personnelle.

5.7. Journal de travail

ETML Section Informatique

Séquence	34		Date: vendredi, 13 mars 2026	Après-midi
Tâche	**Tranche (5min)**		**Explications: qu'est-ce qui se fait et comment ?**	**Liens, références, ...**
Doc - Git	15	1h 15 min	Finalisation du README et dossier livrable.	
Doc - Journal de travail	3	15 min		
Meeting	6	30 min	Evaluation du projet avec la grill d'évaluation TPI	
Doc - Rapport	12	1h	Préparation pour la présentation	
Total tranche	36	3h		

Séquence	33		Date: vendredi, 13 mars 2026	Matin
Tâche	**Tranche (5min)**		**Explications: qu'est-ce qui se fait et comment ?**	**Liens, références, ...**
Doc - Rapport	6	30 min	Finalisation du chapitre 5 sur le rapport.	
Analyse - scénarios	24	2h	Analyse des résultats, temps realisée + démo	
Doc - Journal de travail	3	15 min		
Doc - Rapport	9	45 min	Conception du PowerPoint pour la présentation	
Total tranche	42	3h 30 min		

Séquence	32		Date: mercredi, 11 mars 2026	Après-midi
Tâche	**Tranche (5min)**		**Explications: qu'est-ce qui se fait et comment ?**	**Liens, références, ...**
Doc - Journal de travail	3	15 min		
Test - Unitaire	12	1h	Tests unitaires adaptés au projet	
Doc - Rapport	21	1h 45 min	Chapitres 3 et 4 terminés. A vérifier et améliorer résumé du	
Total tranche	36	3h		

Séquence	31		Date: mercredi, 11 mars 2026	Matin
Tâche	**Tranche (5min)**		**Explications: qu'est-ce qui se fait et comment ?**	**Liens, références, ...**
Test - Unitaire	12	1h	Modifications apportées au test d'intégration destiné au	
Doc - Journal de travail	3	15 min		
Doc - Rapport	21	1h 45 min	Structure des tests et tableaux complétés. Avancement au	
Total tranche	36	3h		

Séquence	30		Date: lundi, 9 mars 2026	Après-midi
Tâche	**Tranche (5min)**		**Explications: qu'est-ce qui se fait et comment ?**	**Liens, références, ...**
Doc - Journal de travail	3	15 min		
Doc - Rapport	24	2h	dossier de réalisation et conclusions en cours	
Analyse - scénarios	6	30 min	Diagrammes du dossier de réalisation : Backend, Frontend	
Total tranche	33	2h 45 min		

Séquence	29		Date: lundi, 9 mars 2026	Matin

ETML Section Informatique

Tâche	Tranche (5min)		Explications: qu'est-ce qui se fait et comment ?	Liens, références, ...
Doc - Journal de travail	4	20 min		
Doc - Rapport	20	1h 40 min	Début de dossier de réalisation et conclusions	
Analyse - scénarios	12	1h	Diagrammes du dossier de réalisation : Backend, Frontend	
Total tranche	36	3h		

Séquence	28		Date: vendredi, 6 mars 2026	Après-midi
Tâche	**Tranche (5min)**		**Explications: qu'est-ce qui se fait et comment ?**	**Liens, références, ...**
Test - Unitaire	9	45 min	Test pas intégré dans le code de git car il fait de	
Doc - Journal de travail	3	15 min		
Doc - Rapport	24	2h	Diagramme d'architecture. Présentation d'un schème pour le dossier de réalisation. Prendre des captures d'écran et fournir des références sur l'état actuel de mon projet.	
Total tranche	36	3h		

Séquence	27		Date: vendredi, 6 mars 2026	Matin
Tâche	**Tranche (5min)**		**Explications: qu'est-ce qui se fait et comment ?**	**Liens, références, ...**
Test - Unitaire	8	40 min	Ajouter des tests en local à essayer plus tard.	
Test - corrections	6	30 min	Bugs trouvé et effacer de code supplémentaire	
Doc - Journal de travail	3	15 min		
Doc - Rapport	19	1h 35 min	Finalisation du chapitre dossier de conception et des	
Absences - imprévus	6	30 min	Meeting avec mon chef de TPI	
Total tranche	42	3h 30 min		

Séquence	26		Date: jeudi, 5 mars 2026	Après-midi
Tâche	**Tranche (5min)**		**Explications: qu'est-ce qui se fait et comment ?**	**Liens, références, ...**
Doc - Journal de travail	3	15 min		
Meeting	7	35 min	Présentation de mon projet sous forme d'une petite démo fonctionnelle. Corrections et améliorations possibles pour	
Doc - Rapport	16	1h 20 min	Ajouter les suggestions de ma cheffe d'équipe dans le	
Test - corrections	10	50 min	Corrections de bugs et suggestion de ma cheffe du projet.	
Total tranche	36	3h		

Séquence	25		Date: jeudi, 5 mars 2026	Matin
Tâche	**Tranche (5min)**		**Explications: qu'est-ce qui se fait et comment ?**	**Liens, références, ...**
Doc - Journal de travail	3	15 min		
Doc - Rapport	30	2h 30 min	Modifications apportées à la documentation, aux images et aux schémas avec légendes et mises au point.	
Test - corrections	9	45 min	Rédaction des tests actuels	
Total tranche	42	3h 30 min		

ETML Section Informatique

Séquence	24		Date: mercredi, 4 mars 2026	Après-midi
Tâche	**Tranche (5min)**		**Explications: qu'est-ce qui se fait et comment ?**	**Liens, références, ...**
Doc - Rapport	18	1h 30 min	Flux de navigation e et finalisation du chapitre 2 Concept (Analyse fonctionnelle, flux de navigation, tests, maquettes	
Doc - Journal de travail	3	15 min		
Réalisation - Git	6	30 min	Modification du gitignore pour adapter à Cloudinary	
Test - Unitaire	9	45 min	Tests et vérifications de fonctionnalités basique du projet	
Total tranche	36	3h		

Séquence	23		Date: mercredi, 4 mars 2026	Matin
Tâche	**Tranche (5min)**		**Explications: qu'est-ce qui se fait et comment ?**	**Liens, références, ...**
Doc - Journal de travail	3	15 min		
Doc - Rapport	27	2h 15 min	Diagrammes de l'architecture logique et finalisation du chapitre 1 Analyse préliminaire (Introduction, Objectifs,	
Réalisation - intégration mobile–backend	12	1h	System de stockage et partage des images avec Cloudinary. Page de uploadePhoto finalisée	
Total tranche	42	3h 30 min		

Séquence	22		Date: lundi, 2 mars 2026	Après-midi
Tâche	**Tranche (5min)**		**Explications: qu'est-ce qui se fait et comment ?**	**Liens, références, ...**
Doc - Git	3	15 min	Réorganisation des tâches en attente et des tâches	
Doc - Journal de travail	3	15 min		
Doc - Rapport	8	40 min	Diagrammes pour définir les cas d'utilisation et son	
Réalisation - intégration mobile–backend	9	45 min	Réalisation de les pages : contrats selon le rôle et réalisation de contrats.	
Meeting	4	20 min	Première analyse de l'avancement du projet. Examen des	
Total tranche	27	2h 15 min		

Séquence	21		Date: lundi, 2 mars 2026	Matin
Tâche	**Tranche (5min)**		**Explications: qu'est-ce qui se fait et comment ?**	**Liens, références, ...**
Doc - Journal de travail	3	15 min		
Doc - Rapport	8	40 min	Diagrammes pour l'explication du program et ces rôles.	https://app.diagrams.net/
Réalisation - intégration mobile–backend	18	1h 30 min	Realisation de les pages : Catalogue, Profil, se desconnecter, afficher contrats.	
Réalisation - application mobile	18	1h 30 min	Adaptation des frontend pour le backend sur : Catalogue, profil et contrats	
Total tranche	47	3h 55 min		

Séquence	20		Date: 27.02.2026	Après-midi
Tâche	**Tranche (5min)**		**Explications: qu'est-ce qui se fait et comment ?**	**Liens, références, ...**
Réalisation - API principale	7	35 min	Modification des requêtes et de leurs noms pour les adapter au frontend	
Réalisation - backend	7	35 min	Modification de Docker et de ses ports pour l'adapter au	

ETML Section Informatique

Réalisation - intégration mobile–backend	20	1h 40 min	Code pour l'intégration du backend avec le frontend : service/ApiService.cs et ajouter des routes	
Doc - Journal de travail	2	10 min		
Total tranche	36	3h		

Séquence	19		Date: 27.02.2026	Matin
Tâche	Tranche (5min)		Explications: qu'est-ce qui se fait et comment ?	Liens, références, ...
Réalisation - Authentification	9	45 min	systèmes de cryptage de mdp et création de droits d'accès pour les utilisateurs	
Réalisation - API principale	9	45 min	Validation d'une liste de requets utilisées pour l'utilisation de l'application	
Doc - Journal de travail	3	15 min		
Réalisation - intégration mobile–backend	21	1h 45 min	Code pour l'intégration du backend avec le frontend : service/ApiService.cs	
Total tranche	42	3h 30 min		

Séquence	18		Date: jeudi, 26 février 2026	Après-midi
Tâche	Tranche (5min)		Explications: qu'est-ce qui se fait et comment ?	Liens, références, ...
Doc - Journal de travail	2	10 min		
Réalisation - base de données	10	50 min	Modification du db pour l'adapter au backend	
Réalisation - API principale	30	2h 30 min	Code : structure des routes et contrôleurs pour contrats	
Total tranche	42	3h 30 min		

Séquence	17		Date: jeudi, 26 février 2026	Matin
Tâche	Tranche (5min)		Explications: qu'est-ce qui se fait et comment ?	Liens, références, ...
Doc - Journal de travail	3	15 min		
Réalisation - Git	6	30 min	Correction de la structure des fichiers	
Réalisation - API principale	20	1h 40 min	Code : structure des routes et contrôleurs pour photos	
Réalisation - backend	10	50 min	Ajustements avec le docker et sa structure pour avoir accès	
Total tranche	39	3h 15 min		

Séquence	16		Date: 25.02.2026	Après-midi
Tâche	Tranche (5min)		Explications: qu'est-ce qui se fait et comment ?	Liens, références, ...
Réalisation - backend	5	25 min	Amélioration de la structure du docker et du nodejs qui gér	
Réalisation - Authentification	7	35 min	La création et la connexion de compte avec l'API fonctionnent.	
Doc - Journal de travail	2	10 min		
Réalisation - base de données	4	20 min	Correction des noms dans la base de données	
Absences - imprévus	18	1h 30 min		

ETML Section Informatique

Total tranche	36	3h		

Séquence	15		Date: 25.02.2026	Matin
Tâche	Tranche (5min)		Explications: qu'est-ce qui se fait et comment ?	Liens, références, ...
Analyse - modèle de données	4	20 min	Correction dans les modèles MCD	
Réalisation - backend	10	50 min	Code docker compose fonctionnel avec le db	
Réalisation - API principale	15	1h 15 min	Code des routes pour la connexion et la création d'un com	
Doc - Journal de travail	4	20 min		
Absences - imprévus	9	45 min		
Total tranche	42	3h 30 min		

Séquence	14		Date: lundi, 23 février 2026	Après-midi
Tâche	Tranche (5min)		Explications: qu'est-ce qui se fait et comment ?	Liens, références, ...
Doc - Journal de travail	3	15 min		
Analyse - définition des rôles et permissions	3	15 min	Modifications apportées à certaines options des contrats conformément au CDC	
Conception - Structure technique	15	1h 15 min	Conception du backend et utilisation avec Docker	
Réalisation - base de données	9	45 min	Code db : photographer	
Meeting	3	15 min	Une autre perspective sur les rôles et les contrats	
Total tranche	33	2h 45 min		

Séquence	13		Date: lundi, 23 février 2026	Matin
Tâche	Tranche (5min)		Explications: qu'est-ce qui se fait et comment ?	Liens, références, ...
Réalisation - backend	6	30 min	Intégration de Docker et conteneurs	
Réalisation - base de données	12	1h	Code SQL et l'intégrer pour Docker	
Analyse - définition des rôles et permissions	6	30 min	Différencier les informations et les rôles nécessaires pour l'API	
Total tranche	24	2h		

Séquence	12		Date: 13.02.2026	Après-midi
Tâche	Tranche (5min)		Explications: qu'est-ce qui se fait et comment ?	Liens, références, ...
Doc - Journal de travail	3	15 min		
Analyse - modèle de données	9	45 min	Modifications réalisé après un feedback pour améliorer	
Conception - Backend	12	1h	Conception de la manière dont le backend et ses besoins p	
Réalisation - backend	12	1h	Code et arborescence des ficher	
Total tranche	36	3h		

Séquence	11		Date: 13.02.2026	Matin
Tâche	Tranche (5min)		Explications: qu'est-ce qui se fait et comment ?	Liens, références, ...
Doc - Journal de travail	3	15 min		

ETML Section Informatique

Analyse - modèle de données	15	1h 15 min	Recherche d'information pour améliorer le MCD	
Conception - Interfaces principales	6	30 min	Petits modifications sur les UX Figma	
Meeting	3	15 min		
Réalisation - Frontend	15	1h 15 min	J'ai ajouté la page Valider Contrat	
Total tranche	42	3h 30 min		

Séquence	10		Date: jeudi, 12 février 2026	Après-midi
Tâche	Tranche (5min)		Explications: qu'est-ce qui se fait et comment ?	Liens, références, ...
Doc - Journal de travail	3	15 min		
Réalisation - Frontend	20	1h 40 min	Page de partage de photo OK. Changement de tous les	
Conception - Interfaces principales	9	45 min	Changement de Coulours, améliorer les centre et positions des objets, ajouter Coulours palette	
Meeting	4	20 min	Montrer avance du projet + corrections sur UX	
Total tranche	36	3h		

Séquence	9		Date: jeudi, 12 février 2026	Matin
Tâche	Tranche (5min)		Explications: qu'est-ce qui se fait et comment ?	Liens, références, ...
Doc - Journal de travail	4	20 min	Ajouter le "réalisation - frontend" sur le planning	
Réalisation - Frontend	14	1h 10 min	Page de photo, comment ajouter la fonctionnalité de camera, et ajouter photo des mon galerie. Page de	
Réalisation - application mobile	12	1h	Améliorations de pages	
Conception - Interfaces principales	12	1h	Améliorations sur interfaces et boutons	
Total tranche	42	3h 30 min		

Séquence	8		Date: 11.02.2026	Après-midi
Tâche	Tranche (5min)		Explications: qu'est-ce qui se fait et comment ?	Liens, références, ...
Doc - Journal de travail	3	15 min		
Réalisation - Frontend	20	1h 40 min	Pages catalogue, contrats, et profil en mode basique OK	
Réalisation - application mobile	10	50 min	Problèmes de compatibilité avec Net9.	
Doc - Git	3	15 min	Ajouter une gitignore	
Conception - Interfaces principales	6	30 min	Améliorations sur les concepts Figma	
Total tranche	42	3h 30 min		

Séquence	7		Date: 11.02.2026	Matin
Tâche	Tranche (5min)		Explications: qu'est-ce qui se fait et comment ?	Liens, références, ...
Doc - Journal de travail	3	15 min		
Analyse - exigences fonctionnelles	10	50 min	Recherches des images et exemples des interfaces app	
Conception - Interfaces principales	15	1h 15 min	Amélioration sur les maquettes : background blanc, pas des options scroll, plutôt des boutons, remplacer le panier	
Réalisation - Frontend	14	1h 10 min	Pages de registre et login en basic OK	
Total tranche	42	3h 30 min		

ETML Section Informatique

Séquence	6		Date: lundi, 9 février 2026	Après-midi
Tâche	**Tranche (5min)**		**Explications: qu'est-ce qui se fait et comment ?**	**Liens, références, ...**
Analyse - scénarios	6	30 min	Recherche des informations et de exemples pour les	
Meeting	6	30 min	Control de travail : améliorer les maquettes avec de vrais exemples, le MCD à corriger. Planification validé	
Doc - Journal de travail	3	15 min	Corrections sur l'orthographie	
Conception - Interfaces principales	12	1h	Corrections	
Total tranche	27	2h 15 min		

Séquence	5		Date: lundi, 9 février 2026	Matin
Tâche	**Tranche (5min)**		**Explications: qu'est-ce qui se fait et comment ?**	**Liens, références, ...**
Doc - Journal de travail	8	40 min	Correction et création des graphiques sur le rapport	
Doc - Rapport	16	1h 20 min	Ajouter les graphiques et les dessin Figma. Chapitre 1 c'est	
Conception - Interfaces principales	12	1h	Améliorations	
Réalisation - Git	6	30 min	Problèmes de synchronisation	
Total tranche	42	3h 30 min		

Séquence	4		Date: vendredi, 6 février 2026	Après-midi
Tâche	**Tranche (5min)**		**Explications: qu'est-ce qui se fait et comment ?**	**Liens, références, ...**
Doc - Journal de travail	2	10 min	Mis à jour	
Analyse – modèle de données	10	50 min	Création du ficher looping pour modèle MCD et MLD	
Réalisation – application mobile	12	1h	Tester les premiers modifications du projet : couleur, nom, logo, etc...	
Doc - Rapport	6	30 min	Ajouter les taches finis - MCD, MLC, etc... + intro	
Total tranche	30	2h 30 min		

Séquence	3		Date: vendredi, 6 février 2026	Matin
Tâche	**Tranche (5min)**		**Explications: qu'est-ce qui se fait et comment ?**	**Liens, références, ...**
Doc - Journal de travail	3	15 min	Mis à jour	
Conception – interfaces principales	21	1h 45 min	Création des premiers modèles de base de smartphones pa	
Absences - imprévus	9	45 min	Recherche de chef du TPI	
Analyse – modèle de données	9	45 min	Création du ficher looping pour modèle MCD et MLD	
Total tranche	42	3h 30 min		

Séquence	2		Date: jeudi, 5 février 2026	Après-midi
Tâche	**Tranche (5min)**		**Explications: qu'est-ce qui se fait et comment ?**	**Liens, références, ...**
Réalisation – application mobile	2	10 min	Téléchargement des ficher + logiciels	
Doc - Journal de travail	2	10 min	Mis à jour + amélioration du planning	

ETML Section Informatique

Analyse – modèle de données	6	30 min	Début de la modélisation des données, comprendre le CdC	
Réalisation – application mobile	6	30 min	Premier débogage du projet sur l'émulateur, pour tester les utiles	
Conception – interfaces principales	20	1h 40 min	Création des premiers modèles de base de smartphones pa	https://www.figma.com/design/quBVhQXEBcIK9HEsbUr6Xn/Untitled?node-id=0-1&p=f&t=nDHxP1ONHgJ1fjt6-0
Total tranche	36	3h		

Séquence	1		Date: jeudi, 5 février 2026	Matin
Tâche	**Tranche (5min)**		**Explications: qu'est-ce qui se fait et comment ?**	**Liens, références, ...**
Doc - Journal de travail	18	2h30	Préparation du journal de travail + planification + problèmes avec l'Excel	https://github.com/Adrtoledocam/P_APPRO1_ProPhotoStock/tree/main/docs/05_JRNTRVL
Doc - Rapport	2	10 min	Mis en place	https://github.com/Adrtoledocam/P_APPRO1_ProPhotoStock/tr
Doc - Git	10	50 min	Préparation d'une arborescence sur GIT	https://github.com/Adrtoledocam/P_APPRO1_ProPhotoStock
Conception – structure technique	10	50 min	Structuration du code app + téléchargement des fichiers (anciens projets), émulateurs et programmes.	
Meeting	2	10 min	Présentation et lancement du projet	https://github.com/Adrtoledocam/P_APPRO1_ProPhotoStock/bl
Total tranche	42	3h 30 min		